IMPERIAL

In-Context Imitation Learning : Instant Policy ++

Author

Teoh Kai Shun

CID: 01878056

Supervised by

Dr Edward Johns

A Thesis submitted in fulfillment of requirements for the degree of
Master of Science in Computing (AI & ML)

Department of Computing
Imperial College London

2025

Abstract

Recent advancements in Non-Euclidean methods in representation learning, along with their success in visual and language tasks, have motivated many studies to apply Non-Euclidean methods to Robotics. In this paper, we propose a potential improvement for Instant Policy paper using Non-Euclidean methods. This approach leverages the expressiveness of hyperbolic space, as demonstrated via performance comparison with its classical counterpart in a 2D task. Instant Policy is a recent approach to In-Context Imitation Learning within Robotics that has been shown to enable rapid learning of various everyday robot tasks from just 1 or 2 demonstrations.

Plain Language Summary

In many robotic tasks where actions are repetitive, it is natural for an AI agent to learn by copying, and this method of learning is known as Behavioural Cloning. However, classical Behavioural Cloning requires a large amount of data, which can be costly; and learning is done with respect to a singular tasks, meaning that learning is not transferrable. Instant Policy is an upgrade of the Behavioural Cloning method which aims to tackle these two problems with Representation Learning.

Intuitively, Representation Learning can be thought of as the AI agent learning to 'frame' the inputs such that it is in a more useful format for subsequent learning tasks. Representation Learning has seen much growth recently, as it has shown its capability to solve more complex problems. With that, we propose a method of Representation Learning for Instant Policy which is more sophisticated than those used initially

Acknowledgments

I would like to thank my Supervisor, my family and my friends for supporting me through this report.

Contents

List of Acronyms

ICIL In-Context Imitation Learning

IL Imitation Learning

GNN Graphical Neural Network

GAT Graphical Attention Network

PMA Product Manifold Attention

List of Figures

List of Tables

1 Introduction

Contents

1.1 Brief Overview

In this paper I introduce an innovative approach of using Hyperbolic embeddings both as a second channel for information propagation (for cleaner information propagation) and as a restriction that explicitly conditions my action predictions based on given demonstrations (for ICIL). Our approach is an extension to Instant Policy [1], which frames ICIL as a graph generation problem. During training we aim to learn the desired denoising actions through supervised-learning, via MSEloss, and during evaluation, we utilise a DDIM scheduler to denoise a random action to derive desired action. Throughout the paper, I have written footnotes (where I express personal thoughts) which I hope that readers can use to better understand my thought process throughout the paper. Figure 1.1 shows a high-level overview of the entire project.

Chapter 2 elaborates on the current state of the art for ICIL and Non-Euclidean methods. Chapter 3 goes through preliminaries that will be important for readers to know before going to methodologies in Chapter 4. Given that this method is motivated and largely similar to [1], I have

1

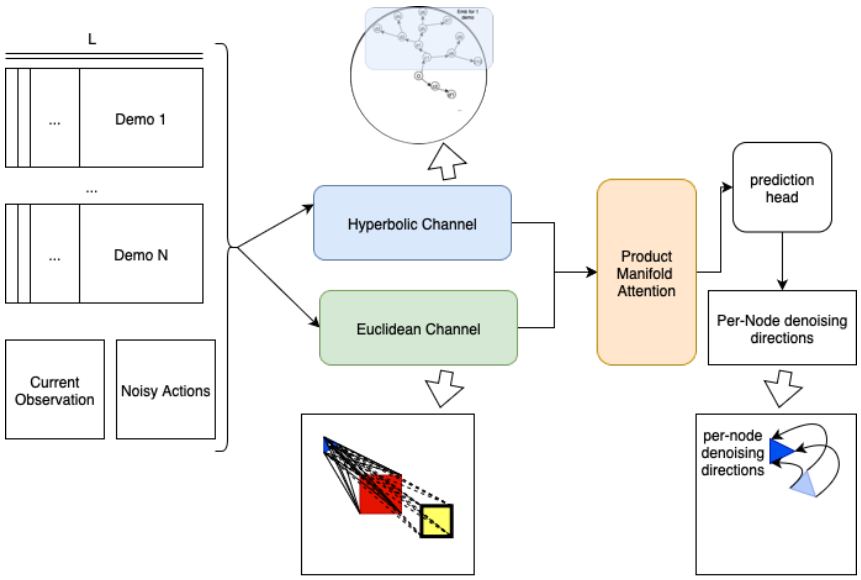


Figure 1.1: High-level overview of Agent

baselined my model Instant Policy in Chapter 5. I conclude with objective evaluation of my model against Instant Policy, and follow up with potential future directions.

1.2 In-Context Imitation Learning

In-Context Imitation Learning (ICIL) within robotics is a relatively young study that has been motivated by the recent success of ICIL in language and visual tasks. The belief behind such motivation is that like visual and language tasks, robotic tasks do possess some underlying semantics or Context. Since ICIL has been shown to enable models to 'extract' this underlying semantic information for visual and language tasks, ICIL should be able to capture the underlying semantic within robotic tasks, and utilise it for predictions.

In addition to the potential success, In-Context Imitation Learning within robotics can potentially revolutionise the use of robots. In its most successful state, whereby In-Context Imitation Learning is capable of learning new complex tasks from only 1 demonstration, its high learning capability will make it an extremely flexible worker. For example, in a packaging company, where packages come in different sizes and shapes, In-Context Imitation Learning can learn to package unseen packages with just demonstrations, removing the need for retraining. More generally, tasks that are laborious and require some extent of generalisability will benefit largely from ICIL within Robotics. This then makes it a very promising study to look into.

1.3 Non-Euclidean Methods

Non-Euclidean Methods stem from a long study of geometry and space, and have recently seen much application in machine learning due to the rise of Representation Learning. Motivated by the success of Euclidean embeddings in machine learning tasks like computer vision and language models, researchers are studying the effectiveness of Non-Euclidean Embeddings. By and large, Hyperbolic embeddings have matched or outperformed their Euclidean counterparts in terms of memory efficiency and accuracy [2][3]. They have also been successfully applied to robotics, from robot designing [4], to trajectory generation [5]. However, in the case of few-shot learning, [6] has shown that Hyperbolic embeddings do not outperform their Euclidean counterparts at high dimensions as it faces numerical hurdles during optimisation.

This essentially means that Non-Euclidean methods, when applied appriopriately, can enable memory-efficient, accurate representations of underlying semantics that we are trying to extract from demonstrations. This can make our model's action prediction be more in line with the demonstration given, which is the desired behaviour for an In-Context Imitation Learner. As such, in this paper we look to effectively apply Hyperbolic embeddings in ICIL within robotics.

1.4 Contributions

The contributions of this paper can be summarised as follows:

1. **Dual-channel architecture (Euclidean + Hyperbolic).** Euclidean embeddings are employed to capture local spatial geometry, while Hyperbolic embeddings represent the hierarchical semantics of demonstrations. This separation addresses the limitation of prior ICIL methods, such as Instant Policy, which rely on a single embedding space to model both.

2. **Product manifold attention for geometry-aware fusion.** The two embedding channels are combined using a product manifold attention mechanism. This design preserves the complementary inductive biases of Euclidean and Hyperbolic spaces, enabling effective alignment between current observations and demonstrations.

3. **Use of Sarkar's constructor for semantic embeddings.** To construct Hyperbolic embeddings without costly optimisation, we employ Sarkar's constructor, which provides training-free, low-distortion embeddings of action trees. This ensures that the Hyperbolic

channel robustly captures task hierarchies while remaining computationally efficient. It also gives us control over the growth of Hyperbolic embeddings, thus avoiding the problem of numerical hurdles.

1.5 Notation

Standard notation has been adopted in the Thesis, most of which is defined in this section and used throughout the remainder of the Thesis. When new notation, not included in this section is introduced, this is defined in the relevant parts of the Thesis.

The symbol $\mathbb{H}^d$ denotes a Hyperbolic space of d-dimension

The symbol $\mathbb{B}^d$ denotes a Poincaré Ball model of d-dimension

The symbol $\mathbb{R}^d$ denotes a Euclidean space of d-dimension

The symbol $\mathbb{S}^d$ denotes a Spherical space of d-dimension

The symbol $\oplus$ denotes a Mobius Addition

The symbol $\otimes$ denotes a Mobius Scalar Multiplication

The symbol $\otimes_M$ denotes a Mobius Matrix-Vector Multiplication

2 Literature Review

Contents

2.1 In-Context Imitation Learning

2.1.1 Imitation Learning

ICIL is an extension to Imitation Learning (IL), which is a form of supervised learning. IL, or more generally known as Behavioural Cloning, is a learning method where the model is trained to imitate its training data. Formally, the data is trained on (input,output) pairs of data generated by an expert. This allows IL to be able to directly leverage knowledge of the experts, which is implicitly expressed within the data. This allows for easier and faster training compared to traditional models that require complex methodologies to encode priors, to a limited extent, within

the models. Furthermore, IL has been shown to be able to learn complex, nuanced behaviour, which traditional models usually struggle with [7].

Given its efficiency in training and its capacity to capture nuanced behaviours, IL has been widely applied in robotics, where it has demonstrated the ability to acquire policies for complex tasks. However, its practicality is constrained by the substantial requirement for high-quality datasets that reflect real-world evaluation scenarios. Unlike computer vision and natural language processing, which benefit from the abundance of large-scale datasets readily available on the internet, robotics lacks such open, publicly accessible resources. [8, 9] mentions that robotic datasets are often proprietary, costly to obtain, or reliant on labour-intensive human data collection. Simulation offers a partial alternative, but IL models generally fail to generalise across the 'reality gap' between simulated and real environments. This gap is largely unavoidable, as faithfully modelling real-world physics is computationally expensive and inherently complex, thereby creating a trade-off between accuracy, computational efficiency, and applicability to specific real-world scenarios

Lastly, IL usually produces a single-task policy, meaning that for a new task to be learnt, the model has to be retrained from another batch of data that corresponds to the new task. This means that previously collected data cannot be reused, even if they are of high quality. In a nutshell, while IL can be successful in Robotics, the cost of data collection and IL learning principle presents a high barrier of application, making its use limited within robotics.

Recent research has attempted to overcome this challenge by moving towards multi-task imitation learning, with recent approaches converging on representation learning strategies [10] due to the latter's success. In these methods, shared encoders are trained to produce latent embeddings of high-dimensional inputs (e.g., images, point clouds, or proprioceptive states), which can then be used to condition policies across multiple tasks. Such representations allow for partial knowledge transfer between tasks and reduce redundancy in data requirements. However, while these approaches improve data efficiency compared to naively training single-task policies, they remain constrained by several factors. First, [10] conclude experimentally that the success of latent representations is often task-dependent, with embeddings that generalise poorly outside the distribution of demonstrations they were trained on. Second, [9] experimentally shows that multi-task policies trained in this way may still suffer from catastrophic forgetting, where learning a new task degrades performance on previously learned tasks. Third, the reliance on large, diverse demonstration datasets may exacerbate the underlying challenge of human data collection in robotics.

Alternative strategies, such as meta-learning frameworks [11] and hierarchical policies [10] have

also been explored. These methods aim to leverage shared structure across tasks to improve adaptability. Yet, despite promising progress, current approaches often demonstrate limited generalisability when transferred from benchmark simulation environments to real-world robotic platforms [9]. In practice, the multi-task extension of IL therefore remains an open problem, with existing solutions heavily reliant on representation learning pipelines that only partially alleviate the fundamental data inefficiency.

2.1.2 Representation Learning and Few-shot Learning

Representation learning aims to learn some hidden latent variable (of symbolic data) that is capable of retaining underlying information, such that they can be used for predictions. Representation learning has seen much application and success in the recent years. To list a few, [12] have utilised latent variables to capture features within images for segmentation, and more relevantly, [13] have shown that language models (which inherently utilises latent variables) are natural few-shot learners.

One of the contributions of Representation learning is making few-shot learning a possibility. Few-shot learning is a form of learning that attempts to generalise to new tasks given demonstrations of new tasks. With latent variables providing strong representations, models are able to capture task-relevant invariance in input data such that it is able to generalise from just a few demonstrations. Much like representation learning, Few-shot learning models have been shown to be successful in tasks working with symbolic data, like object detection and text classification [14]. This can imply that few shot learning algorithms might perform for robotic tasks, as robotic tasks deals with sequential data (like point-clouds and end-effector grip poses) and as such, robotic tasks may benefit from few-shot learning approaches in a manner analogous to the success seen in symbolic domains, particularly when learning new manipulation skills or adapting to novel objects with minimal demonstration data.

2.1.3 Towards ICIL for robotics

Few-shot learning serves as the bridge between IL and ICIL. By combining the imitation principle from IL with few-shot learning's ability to generalise from limited demonstrations, ICIL attempts to learn a model that can imitate new behaviours from just a few examples.

The key distinction lies in how these approaches handle generalisation. Traditional IL models

learn task-specific policies by directly mapping state-action pairs from training data, without leveraging the underlying semantic structure of the demonstrations [1]. This limits them to reproducing behaviours they have explicitly seen during training.

In contrast, ICIL models learn context-conditioned policies that can adapt their behaviour based on the demonstrations they receive at test time [1]. This is made possible through representation learning, which enables the model to extract and utilise the underlying semantics from demonstrations rather than simply memorising surface-level patterns. By learning to identify task-relevant features and invariances in the demonstration data, ICIL models can understand the intent behind demonstrated actions and generalise this understanding to new situations. This semantic understanding allows ICIL models to condition their action predictions on the context provided by new demonstrations, effectively learning a meta-policy that can rapidly adapt to new tasks. As a result, robots can acquire new skills from just a few human demonstrations, significantly reducing the data collection and training requirements of traditional robotic learning approaches

2.1.4 In-Context Imitation Learning in Robotics

In-context imitation learning (ICIL) in robotics is a relatively young but rapidly developing field. Existing approaches largely adopt either diffusion-based or sequence-modelling frameworks. For example, [1] formulates ICIL as a graph generation problem, where noisy action trajectories are iteratively denoised via per-node updates conditioned on demonstrations. In contrast, [15] frames ICIL as a next-token prediction problem, encoding scenes as tokens, leveraging large sequence models for prediction, and decoding actions from generated tokens. These formulations illustrate the growing trend of casting robot policy learning as a generative modelling problem.

Early results have been promising: state-of-the-art ICIL models demonstrate strong context alignment (the ability to adapt policies to novel tasks by attending over demonstration trajectories and dynamically conditioning action generation). For instance, [1] shows that attention weights between demonstrations and current observations evolve coherently as the agent progresses through a task, enabling effective adaptation without gradient updates.

However, these methods remain far from robust. Their success is tightly coupled to the quality and structure of demonstrations: systematic preprocessing is required to highlight semantic transitions (e.g., waypoints or key interactions), and the absence of such structure can lead to degraded performance in complex or long-horizon tasks. Furthermore, the reliance on perception modules introduces fragility – low-latency errors in segmentation or object detection can propa-

gate downstream, causing cascading policy failures. Finally, current ICIL methods remain largely validated in simulation or constrained setups; transfer to unstructured real-world environments, where demonstrations are noisy, incomplete, or inconsistent, is still an open challenge.

2.1.5 Current Limitation

In summary, while ICIL has shown promising progress in enabling robots to adapt from only a handful of demonstrations, its performance is still tightly constrained by the quality, structure, and semantic richness of those demonstrations. Existing approaches often struggle to disentangle global semantic intent from local spatial details, leading to brittle generalisation in complex or noisy environments. These limitations directly motivate the design of our proposed model: by explicitly separating semantic information (captured in a Hyperbolic channel that encodes tree-like, hierarchical action structures) from spatial information (captured in a Euclidean channel), and then fusing them via a geometry-aware attention mechanism, we aim to improve context alignment and robustness to suboptimal demonstrations beyond what prior ICIL models achieve.

2.2 Non-Euclidean Methods

The use of Non-Euclidean methods within machine learning is brought about by the success of Euclidean representation learning. Representation learning by and large is done in the Euclidean setting [16], and given it success, have motivated researchers to explore Non-Euclidean representation learning for further improvements. Majority of these research are targeted at Hyperbolic embeddings, which I will elaborate below.

2.2.1 Hyperbolic Representation Learning

Hyperbolic embeddings have been shown to be capable of modelling tree-structured symbolic data with significantly lower dimensions [2] and lower distortion than their Euclidean counterparts. [2, 3, 17] have experimentally shown that Hyperbolic embeddings outperform Euclidean embeddings in terms of both computation and accuracy, with [3] even showing that a 2-dimension Hyperbolic embedding will be sufficient in embedding a tree-like structured data, and that this performance cannot be matched by a Euclidean embedding with 50+ dimensions. It has also been widely used to embed knowledge graphs with complex relationships [18]. With its success, many researchers

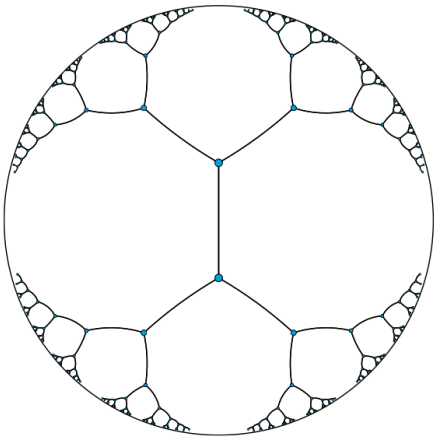


Figure 2.1: A tree-graph being embedded within Poincaré ball model. Nodes are evenly spaced apart and they approach ball boundary, but never reach it, as distance from ball origin to boundary is theoretically infinite. Taken from [2], fig 1.b

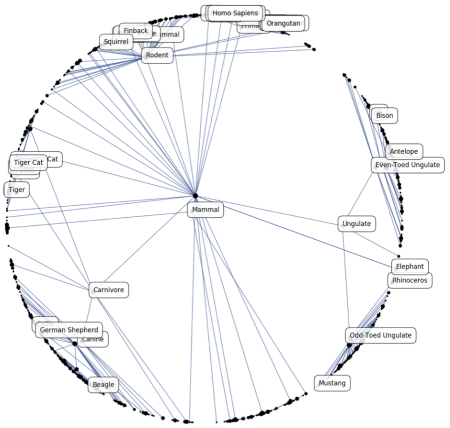


Figure 2.2: An example of real-world data exhibiting tree-structure being embedded by Poincaré ball model. Taken from [2], fig 2.b

have applied Non-Euclidean methods within robotics, and generally they have seen much success. [4] utilises Hyperbolic data inherent tree structure to guide its robotic design search space, and [5] utilises Hyperbolic embeddings to model taxonomy data for targeted trajectory generation.

The difference between Hyperbolic space and Euclidean space lies in their curvature. Euclidean space has zero curvature, meaning its volume grows linearly with radius, while Hyperbolic space has negative curvature, leading to exponential volume growth. This property allows Hyperbolic space to model hierarchies in low dimensions with low distortion. In contrast, Euclidean embedding models, which typically rely on translation [19] and rotation [20] operations, require higher dimensions [17] to avoid excessive compression of the data (when embedding), since Euclidean space lacks the exponential representational capacity of Hyperbolic geometry.

Furthermore, the negative curvature of Hyperbolic space effectively "folds up" the manifold, enabling it to naturally embed hierarchical structures such as trees and taxonomies. In Euclidean space, representing hierarchies leads to severe distortion because the available volume only grows polynomially with radius, while hierarchical structures expand exponentially with depth. Hyperbolic space resolves this mismatch: the exponential growth of its volume matches the branching growth of hierarchies, so nodes at different levels can be placed at increasing radii without overlap or compression. This is due to its negative curvature, which causes geodesics to grow exponentially far apart, resulting in the exponential growth in volume. Empirically, this property has been demonstrated in works such as [2, 17], where Hyperbolic embeddings were shown to preserve se-

mantic hierarchies in knowledge graphs and capture hierarchical similarity in symbolic and visual data. In effect, Hyperbolic geometry allows distance metrics in the manifold to align with the underlying tree-like topologies of the data, yielding low-distortion embeddings that remain compact in dimension. (See Fig 4.2 for visualisation.)

However, it is important to note that this does not mean that Hyperbolic embeddings are in general superior to Euclidean embeddings. Different embedding spaces are suited for different geometric structure: Euclidean being suited for linear flat spaces [16], while Hyperbolic being suited for tree-like structures like knowledge graphs [17]. Ultimately, the representation strength of an embedding is determined by the underlying data structure that it is embedding, and it is important to identify or approximate through analysis the underlying structure of the data before choosing an embedding space. From a more mathematical point of view, Hyperbolic spaces can face optimisation issues. [6] notes that Hyperbolic embeddings might not outperform their Euclidean counterparts, as high dimensional Hyperbolic embeddings tend to saturate at the Poincaré ball boundary, resulting in vanishing gradient, putting learning to a stop.

Hyperbolic embedding models

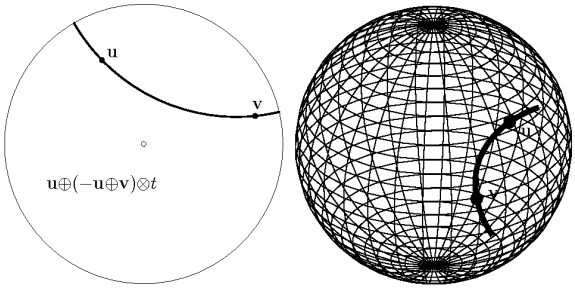


Figure 2.3: Figure shows the geodesic distance of the Poincaré ball model (left) and the Hyperboloid model (right). Taken from [21], fig 1.2

There are many Hyperbolic embedding models, but the majority of the Hyperbolic embeddings methods utilise the Poincaré ball model as it is conformal, mathematically convenient and robust. It has also been shown to be successful experimentally [2]. The mathematical convenience lies it its closed-form distance function, which makes gradients computable and training efficient. Its robustness comes from the fact that all points are bounded within a unit ball, effectively preventing divergence of embeddings during training. While other models like the hyperboloid models also offer these mathematical convenience and robustness, Poincaré models are typically chosen for its interpretability. Since angles in data are preserved within embeddings, and geodesics are tractable (unlike more abstract models like the hyperboloid model), many researchers opt for Poincaré model

so that they can better evaluate their models.

On the other hand, recall that [6] mentions the numerical hurdle for optimisation when using Poincaré ball model in high dimensions. Hyperboloid model in this case might be the better option as it avoids boundary instabilities. [22] shows that Poincaré ball model is far more prone to gradient vanishing and unstable optimization, especially near its boundary. In contrast, hyperboloid model shows much stronger numerical stability during optimization, with gradient updates remain well-defined even for points near effective boundary.

The decision on which Hyperbolic embedding models to use is largely application-dependent. The Poincaré ball model provides good interpretability, allowing one to analyse the embeddings and ensure that the learning is aligned with deployment. For example, geodesic distance within Poincaré ball model, which can be visualised as arcs of the unit ball, can also be utilised as an metric for attention weights between embeddings (queries and keys). By contrast, for a hyperboloid model, geodesic distance are extremely abstract [1]. On the other hand, if learning is deep, and model frequently utilises embeddings for predictions (meaning that gradients gets deep), stability might be preferred over interpretability.

2.2.2 Hyperbolic Neural Network

With the success of Hyperbolic embeddings, researchers furthered the study of Non-Euclidean methods in machine learning by extending classical Euclidean neural network components to its Hyperbolic counterparts. Early attempts include [23, 24] which outlines a transformer that has a self-attention mechanism that is adapted for Hyperbolic space. At a more mature stage, [25, 26] had both outlined the Hyperbolic adaptations for these crucial neural network components. In both papers, mobius transformation is utilised to adapt Linear layers such that Hyperbolic geometry is maintained, with activation functions being applied in tangent space to leverage familiar Euclidean operations while operating in Hyperbolic space. Furthermore, by having an attention mechanism based on Hyperbolic distance and mobius concatenations, models are able to better capitalise on hierarchical relationships captured by Hyperbolic distances, thereby allowing it to perform attention while taking into account this hierarchical relationship. Now, [27] furthers the horizon of Hyperbolic neural network by introducing a Hyperbolic Temporal Graph Network (HTGN) that is capable of embedding a temporal graph. The graph network is equipped with a Hyperbolic

[1] This is partially why I chose Poincaré ball model, so I could show the attention mechanism between demonstrations and current observations. Furthermore, since I had control over the growth of my Hyperbolic embeddings, I am convinced that this is not an issue.

Temporal Attention layer for temporal awareness, Hyperbolic Graph Neural Network for local neighbourhood awareness and a Hyperbolic Gated Recurrent Unit for memory, and experimentally, it has been shown to significantly outperform state-of-the-art models like dySAT[28].

2.2.3 Learning with Mixed Curvature

While most Non-Euclidean embedding methods focus on Hyperbolic spaces, some approaches adopt a more comprehensive strategy by using product manifolds [16, 16]. Rather than constraining embeddings to purely Hyperbolic, spherical, or Euclidean spaces, these methods construct embedding spaces from combinations of different geometric components, essentially creating hybrid spaces with heterogeneous curvature properties [16]

This approach stems from the recognition that certain real-world data often contains both hierarchical structures (naturally suited to Hyperbolic geometry) and cyclical patterns (better captured by spherical geometry) simultaneously [29, 17]. For example, in fine-grained tasks such as threading a needle or inserting a key, the action sequence often involves oscillatory adjustments for alignment and exhibits a hierarchical structure, with movements becoming progressively finer as alignment improves. Since embedding quality fundamentally depends on how well the geometric properties of the embedding space match the intrinsic structure of the data, using mixed-curvature product spaces allows for more flexible geometric alignment [16, 2].

Experimentally, [16] has shown that their models utilising dynamic-curvature space have outperformed multiple state-of-the-art models utilising constant-curvature space in Knowledge Graph Completion tasks on multiple recognised datasets. However, it is important to note that Product Manifolds embeddings should not be treated as a one-for-all solution. For the case of Knowledge graph, where rich and complex relations exist between entities, using product manifold will not be an overkill, while for straightforward data whose relations can be approximated by a linear grid [2], Euclidean embeddings will usually suffice. It is also important to acknowledge that part of the performance improvements comes with significant computation cost incurred by Non-Euclidean components [26, 30]. Selecting the correct number of manifold and their dimension is also a key challenge, as they need to reflect the underlying relations of the data[3]. Fortunately, [16] offers a heuristic estimator for these hyper-parameters.

[2] like point-clouds, personally I think [1] showed that semantic information within point-clouds can be sufficiently encoded with Euclidean embeddings

[3] This is not a challenge faced by us, given the simplicity of our model.Since [3] has shown that a $\mathbb{B}^2$ can sufficiently embed a hierarchical structure, and in Section 3.1 we propose semantics within actions to be hierarchical in 2D, we utilise a $\mathbb{B}^2$ Hyperbolic space to embed actions. For Euclidean embeddings, we estimate our dimensions from those use in [1]. This will make more sense in Chapters 3& 4.

2.2.4 Current Limitation

Taken together, the literature on Non-Euclidean methods highlights both the expressive power and the practical challenges of Hyperbolic and mixed-curvature embeddings. Hyperbolic spaces naturally capture hierarchical semantics but can suffer from optimisation instabilities [27, 22], while Euclidean embeddings remain stable but fail to represent tree-like structures compactly. Prior work rarely addresses how to combine these complementary strengths in the context of robotic imitation learning. Our approach is designed precisely to bridge this gap: we exploit Sarkar's constructor and Hyperbolic GNNs to embed action hierarchies with low distortion, while retaining a Euclidean channel to anchor these semantics in stable spatial features. By unifying the two via product-manifold attention, we align demonstrations with current observations in a way that respects both geometric fidelity and semantic hierarchy.

3

Preliminaries

Contents

This chapter presents the necessary preliminaries to support the understanding of the methodology discussed in the subsequent chapter. It provides the theoretical background and technical detail required, thereby allowing the next chapter to remain focused on the motivation, rationale, and the *how* of the proposed implementation. Specifically, this section introduces the Instant Policy framework and Hyperbolic components that I will be using in my proposed models. Appendix D establishes the mathematical foundations of Hyperbolic space that underpin the methods developed later, and I highly recommend readers to go through it briefly at least.

3.1 Defining semantics within actions

Motivation. Before covering prior work and the mathematical tools, I first clarify what I mean by "semantics within actions", since this viewpoint motivates several design choices in Chapter 4.

Conceptually, any human task can be decomposed into a sequence of commands. These commands, in turn, correspond to waypoints that guide the agent through intermediate states required to accomplish the task. Moving through such waypoints can thus be viewed as transitioning through semantic states.

In a 2D setting, these semantic transitions typically consist of rotations (changes in orientation) and state-changes (changes in agent condition or interaction with the environment). A semantic transition can therefore be modelled by either orientation/state changes [1].

This naturally raises the question: how can such transitions be effectively represented so that they capture the underlying semantics of actions? For example, a demonstration of length L may involve only K semantic transitions, where generally $L > K$. The mapping between frames and semantic states is therefore not one-to-one, which motivates the need for clustering demonstration frames such that each cluster corresponds to a semantic unit.

A second challenge is that waypoints can grow exponentially: any waypoint can always be subdivided into finer waypoints. At first this seems problematic, but it is in fact a useful property—it reflects the hierarchical nature of actions. Complex actions can always be decomposed into constituent sub-actions, and conversely, sub-actions must recombine to form the whole.

This hierarchical structure suggests that we can recursively cluster action frames based on orientation changes and state transitions at decreasing angular granularities. These clusters naturally form a tree, where each parent of the tree at depth l denotes a semantic state at level l. Crucially, Hyperbolic space (via the Sarkar constructor) provides a geometry well-suited to embedding such hierarchical trees with low distortion.

Thus, by fixing a set of granularities, we can construct tree representations from demonstrations: each demonstration becomes a sequence of nested curves in which angles evolve monotonically within local windows. Embedding these trees in Hyperbolic space yields symbolic Hyperbolic embeddings that explicitly capture the semantic hierarchy of actions.

[1] I pick state change as a priority condition as it seems to contain more 'semantic shift'.

3.2 Instant Policy

In this section, I will only go through the parts of Instant Policy that I have adapted for my model. I first go through how Instant Policy constructs graph representation for a given scene or action, followed by the attention mechanism utilised by Instant Policy. Lastly, I introduce the concept of Pseudo Demonstration, which is used as a source of infinite training data. For a more comprehensive overview, I recommend reading the paper [1].

3.2.1 Graphical Representation for Observations and Actions

In instant policy, observations include point clouds and agent keypoints (in world frame). Agent keypoints are predefined, and in Instant Policy's case, they are the end-effector point on the robot gripper.

Firstly Instant Policy processes point clouds by passing them through a pre-trained SetAbstraction layer [31, 32] to produce M centroids with spatial features[2]. These features then make up the object embeddings, and by treating the corresponding centroid as a scene node, we reduce the set of point clouds into a set of scene nodes and features. For the agent nodes, Instant policy distinctly creates embeddings for each keypoint. These embeddings are then concatenated with embeddings representing agent's state. Finally, by treating each keypoint as agent nodes, we connect each agent node to all scene nodes. Each edge is given features shown in equation (3.1) as per [33] to better capture high-frequency changes and increase precision. This is also so that during Hetero-Graph Attention Transformer layer (see Section 3.2.2), information from scene nodes are able to propagate into agent features. With this construction, we get *Local Graph*, which aims to best capture spatial relations within each observation and aggregate it at agent nodes.

$$e_{ij} = [sin(2^0\pi(p_j - p_i)), cos(2^0\pi(p_j - p_i))...sin(2^{D-1}\pi(p_j - p_i)), cos(2^{D-1}\pi(p_j - p_i))] \quad (3.1)$$

With *Local Graph*, it is easy to represent actions. For a given action a_t taken at timestep t from observation o_t, we take the *Local Graph* constructed from observation o_{t+1}, which is the observation after taking action a_t from observation o_t. These are called *Action Graph*. While it is intuitive to update agent position with actions, [1] recommended to update object scene node positions with

[2] M is a Hyper-parameter. It can be thought of as the number of centroids sampled.

action, so that spatial information can be recalculated. This is done with the purpose to better match the spatial information from demonstrations, so that the model can be more aligned with demonstrations.

3

3.2.2 Hetero-Graph Attention Layer

The Hetero-Graph Attention Layer is one of the key components of Instant policy, as it not only helps aggregate important spatial information into the agent nodes, but it also plays a role in aligning the current observation with given demonstration, and aligning action with the current observations. Experimentally, it has been shown by [1] that Hetero-Graph Attention Layer can enable alignment of current observation with demonstrations. As such, it is a powerful tool that I have decided to use in my model.

The Hetero-Graph Attention Layer can be described in 2 equations. Given that each node in a graph has feature $\mathcal{F}_i$, this layer updates the feature with:

$$\mathcal{F}_i' = W_1\mathcal{F}_i + \sum_{j\in\mathcal{N}(i)} att_{i.j}(W_2\mathcal{F}_j + W_5 e_{i,j}) \tag{3.2}$$

$$where: \ att_{ij} = softmax(\frac{(W_3\mathcal{F}_i)^T(W_4\mathcal{F}_j + W_5 e_{ij})}{\sqrt{d}}) \tag{3.3}$$

where d is the per-head key dimension.

Within Instant Policy, this attention mechanism is applied to (1) aggregate scene information[3] into agent nodes for each scene, (2) align the current observation with given demonstrations[4] (through attention mechanism with aggregated agent nodes information after (1).) and lastly (3) to propagate information from current agent node to post-state agent nodes in **Action Graphs**[5]. However, within our method, we only utilise this mechanism for only (1).

3.3 Sarkar's Constructor

Sarkar's Constructor is a combinatorial and training-free procedure to embed any finite tree $T = (V, E)$ into Poincaré Ball $\mathbb{B}^d$ [34]. Experimentally, [4, 30] has shown that these resulting embeddings have arbitrarily low distortion, and have very strong representation power. [30] notes a 0.989

[3] Rho in Instant Policy
[4] Phi in Instant Policy
[5] Psi in Instant Policy

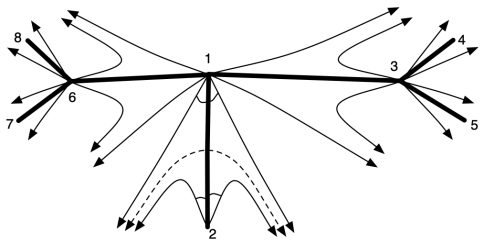


Figure 3.1: Figure visualising how Sarkar's Constructor embeds a tree. As seen, branches expands away from each other to ensure that nodes are evenly-spaced. Taken from [34]

mean-average precision[6] when constructing embeddings on WordNet(has a tree-like structure), and consistently outperforms baselines models when embedding datasets with tree or tree-like structure. As such, given symbolic data with underlying tree structure, embedding them with Sarkar's Constructor will give computation benefits from the lack of training, while remaining representative.

3.3.1 Generation Procedure

The Sarkar's Constructor takes in a tree and recursively embeds the tree nodes, from root to leaves. It takes in at each recursion step a node a and its parent b, and recursively embed children of $a : c_1...c_{deg(n)-1}$, with respect to b's Hyperbolic embedding and a's Hyperbolic embedding. This happens with a 3 step process. Given that embeddings of a and b are $f(a)$ and $f(b)$ respectively, we first reflect both of them across a geodesic using a circle inversion so that $f(a)$ is mapped onto origin 0 and $f(b)$ is mapped onto some point z [30]. Then children of a are placed to vectors $z_1....z_{deg(a)-1}$ such that they are equally spaced (with respect to Hyperbolic distance) around a circle with radius τ in Hyperbolic space, and are maximally separated from the reflected parent node embedding z. In Euclidean space, the radius will be $\frac{e^\tau-1}{e^\tau+1}$. All points are then reflected back across the geodesic, and the process repeats again with the one of the children c_i and parent a. The recursion starts by placing the root at the origin and its children in a circle around it. To get vectors z_i, [30, 34, 4] has utilised this equation:

$$z_i = \frac{e^\tau - 1}{e^\tau + 1} \cdot (cos(\theta + \frac{2\pi i}{deg(a)}), sin(\theta + \frac{2\pi i}{deg(a)})), \ \forall i \in [0 : deg(a) - 1] \tag{3.4}$$

where θ is the angle of the reflected parent node z from x-axis in the plane. Pseudocode for

[6] mean-average precision (mAP) is a common metric to compare embeddings quality within Representation Learning. It measures the amount of 'distortion' caused by the embeddings, on the data.

Sarkar's Constructor and more details on properties of embeddings from Sarkar's Constructor can be found in Appendix C.

3.3.2 Why Sarkar's Constructor?

Learning Hyperbolic embeddings from scratch can be very computationally expensive [26]. Given that we are going to be embedding each scene (of demonstration and current observation and post-state observations), it will not be wise to learn these embeddings from scratch, on top of the learning that we are already performing. Furthermore by using Sarkar's Constructor, we **explicitly** condition the current observations and demonstrations to action predictions, as Sarkar's embeddings are constructed from them. Furthermore, by defining the angular granularities being used for clustering, the depth of the constructed tree can be controlled, effectively leading to robust optimisation even with Poincaré ball model. Finally, because Sarkar's constructor can embed hierarchical data with low distortion, it yields good representations even when working with approximations of the true underlying mixed-smooth curves. In this framework, the demonstrations serve as approximations of these underlying curves, while Sarkar functions solely as the embedding mechanism rather than contributing to the approximation itself.

3.4 Mobius Operation

In this section, I cover the mathematical tools that will allow us to apply Euclidean functions within Hyperbolic space. Specifically, I first touch on a 2-way, 1-to-1 mapping that allow us to move between Euclidean and Hyperbolic space while preserving geometry of embeddings. This allows us to apply Euclidean functions on Hyperbolic without breaking geometry. Secondly, I touch on aggregation method within Hyperbolic space, which will underpin Hyperbolic value aggregation in Section 4.4.4.

3.4.1 Expmap and Logmap

The logarithmic map $log_0 : \mathbb{B}^n \to T_0\mathbb{B}^n$ maps points from the Poincaré ball model to the tangent space at the origin [26]. It is given by:

$$\log_0(x) = \frac{2 \operatorname{artanh}(\|x\|)}{\|x\|} x, \tag{3.5}$$

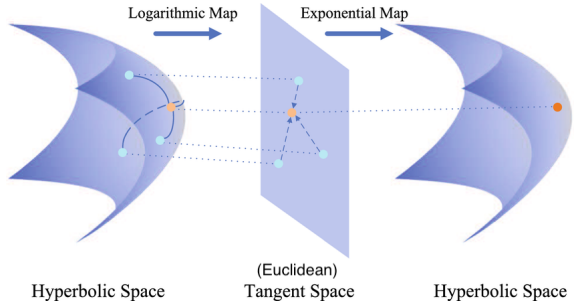


Figure 3.2: Figure depicts how expmap and logmap links Hyperbolic Space and Euclidean Space. Taken from [35]

Conversely, the exponential map $exp_0 : T_0\mathbb{B}^n \to \mathbb{B}^n$ brings points back to the manifold [26]. It is given by:

$$\exp_0(v) = \tanh\left(\frac{\|v\|}{2}\right)\frac{v}{\|v\|}. \tag{3.6}$$

Additionally, from their definitions, exp_0 and log_0 are 1-to-1 mappings of the manifold and tangent space at origin, and are well defined for all $v \in \mathbb{R}^d$ and all $x \in \mathbb{B}^d$. This makes them a mathematically robust to link Euclidean space and Hyperbolic space. Effectively these maps enables us to utilise Euclidean activation functions and normalisation [26]. We simply apply these function by tangent space with log_0 and then map it back to manifold with exp_0. Formally they are:

$$ReLU^{\mathbb{B}}(x) = exp_0(ReLU(log_0(x))) \tag{3.7}$$

$$LayerNorm^{\mathbb{B}}(x) = exp_0(LayerNorm(log_0(x))) \tag{3.8}$$

These extends to most vanilla Euclidean functions too. For any Euclidean functions F acting on Hyperbolic variable x, exponential map and logarithmic map are used as such to retain Hyperbolic geometry:

$$F^{\mathbb{B}}(x) = exp_0(F(log_0(x))) \tag{3.9}$$

Convention. To be really clear, throughout this work we set $c = 1$, hence the closed forms above use the standard Poincaré ball radius and factors.

3.4.2 Karcher Mean

The aggregation function usually aims to aggregate node features from a Graphical Neural Network (GNN). However, for GNN working with Hyperbolic embedding, Euclidean aggregation functions can break Hyperbolic geometry, resulting in meaningless aggregated features. As such there needs to be a geometry-preserving method to aggregate Hyperbolic features. Karcher Mean is one of these methods, and it aims to extends the concept of weighted average to Non-Euclidean spaces while preserving geometry [16].

The Karcher mean is defined as the minimiser of the sum of squared Riemannian distance [36]. Given points $x_1, x_2, ...x_n$ in a Riemannian Manifold $(\mathbb{B}, g)$, the Karcher Mean is defined as[37]:

$$\bar{x} = \arg\min_{x \in \mathbb{M}} \sum_{i=1}^{n} w_i \cdot d_g(x, x_i)^2 \tag{3.10}$$

where $\{w_1...w_n\} = W$ are learnable weight parameters. By minimising Hyperbolic distance, aggregation is done without breaking Hyperbolic geometry. The embeddings, or more generally the Hyperbolic space, retains its continuous tree-like structure, with the aggregated feature being 'hierarchically-aware' as a result.

To learn weights W, we define the gradient of weights at point x to be below. Here $log_x(x_i)$ is defined as the logarithmic map that maps point x_i to the tangent space at x.

$$\nabla F(x) = -2 \sum_{i=1}^{n} w_i \cdot log_x(x_i) \tag{3.11}$$

where

$$log_x(y) = \frac{x}{1 - ||x||^2} artanh(|| - x \oplus y||) \frac{-x \oplus y}{|| - x \oplus y||}$$

Then with the gradient, one can take a Riemannian gradient descent by updating x' to be:

$$x' = exp_{x'}(-\alpha \nabla F(x)) \tag{3.12}$$

where

$$exp_x(v) = x \oplus tanh(\frac{||v||_x}{||2||}) \frac{v}{||v||}$$

For a more mathematical review and explanation of Karcher Mean, one can check out [36, 37]. Specifically, [37] mentions a list of properties of Karcher Mean that justifies its effectiveness as a

general Riemannian mean.

3.5 Hyperbolic GNNs

3.5.1 Hyperbolic GNN and Hyperbolic Graph Attention Networks

Before going through this section, I highly recommend readers to read up Appendix D, especially Section 3.4.1, which talks about utilising exponential map and logarithm map to extend Euclidean operations into Hyperbolic space.

Hyperbolic Graphical Attention Network (GAT) can be thought of as GAT that acts on the tangent space of given node [38, 39]. The Hyperbolic GAT that I have used is a slightly augmented version of the one proposed in [38]. Specifically I have utilised learnable attentions as opposed to calculating attention scores with negative distance; and have added a Depth-dependent linear gate to modulate my attention score to make learning robust (since they are learnt). For succinct explanation, I start by providing the general workflow of Hyperbolic GAT with a pseudocode below, and followed by a brief explanation of the workflow. Then, I explain the important steps that maintains Hyperbolic geometry.

Algorithm 1 Hyperbolic Graph Attention (Learnable Attention)

Require: Directed graph $G = (V, E)$, node states $\{x_i \in \mathbb{B}_c^2\}_{i \in V}$, curvature $-c < 0$ $(c > 0)$, scorer $s : \mathbb{R}^2 \to \mathbb{R}$, step size $\eta > 0$.
(Optional) depth embedding giving per-node scale k_i and angle θ_i.
Ensure: Updated node states $\{x_i^{\text{new}}\}$ on $\mathbb{B}_c^2$.

1: **for all** nodes $i \in V$ **do**
2: $M_i \leftarrow \emptyset$ ▷ buffer for incoming edge messages in $T_{x_i}\mathbb{B}_c^2$
3: **for all** incoming neighbours $j \in \mathcal{N}^{\text{in}}(i)$ with edge $j \to i$ **do**
4: $v_{ij} \leftarrow \log_{x_i}(x_j)$ ▷ move x_j to the tangent space at x_i
5: $\ell_{ij} \leftarrow s(v_{ij})$ ▷ attention logit for edge $j \to i$ (2-layer MLP + GELU)
6: add (v_{ij}, ℓ_{ij}) to M_i
7: **end for**
8: **# segment softmax at receiver i for numerical stability**
9: $m_i^\star \leftarrow \max\{\, \ell_{ij} : (v_{ij}, \ell_{ij}) \in M_i \,\}$ ▷ if $\mathcal{N}^{\text{in}}(i) = \emptyset$, skip to line 16
10: $\alpha_{ij} \leftarrow \exp(\ell_{ij} - m_i^\star) / \sum_{(v_{ik}, \ell_{ik}) \in M_i} \exp(\ell_{ik} - m_i^\star)$
11: **# tangent aggregation at x_i**
12: $m_i \leftarrow \sum_{(v_{ij}, \ell_{ij}) \in M_i} \alpha_{ij}\, v_{ij} \in T_{x_i}\mathbb{B}_c^2$
13: **# depth gate in $T_{x_i}\mathbb{B}_c^2$:**
14: $\tilde{m}_i \leftarrow k_i R_{\theta_i} m_i$, where $R_{\theta_i} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i \\ \sin\theta_i & \cos\theta_i \end{bmatrix}$ ▷ Radial-Dilation + Rotation along origin in Hyp space
15: **(Riemannian update on the manifold)**
16: $x_i^{\text{new}} \leftarrow \exp_{x_i}(\eta\, \tilde{m}_i)$ ▷ use $\tilde{m}_i = m_i$ if gate is off
17: **end for**

Algorithm 1 illustrates a single attention-based message passing step on the Poincaré ball. For each incoming edge e_{ji}, the neighbour embedding x_j is mapped to the tangent space at the receiver x_i via the logarithmic map. Attention scores are computed from the tangent vectors and normalised per receiver using a segment-wise softmax. The resulting weighted tangent vectors are aggregated into m_i, then modulated by a depth-dependent linear gate. Finally they are mapped back to the manifold through the exponential map with step size η.

Within the entire pseudocode, the Hyperbolic geometry preserving steps are the logarithmic and exponential mappings (see Section 3.4.1 for more details).

Depth–dependent linear gate. Within our Hyperbolic GAT, we require a lightweight mechanism to modulate aggregated attention weights according to the hierarchical structure of the demonstrations[7]. To this end, we introduce a *depth–dependent linear gate* that acts directly in the Hyperbolic space.

Given points $x \in \mathbb{B}_c^2 \subset \mathbb{R}^2$ with corresponding depths $d \in \{0, \dots, D-1\}$, the gate performs two operations in the tangent space at the origin: a depth–wise radial rescaling (inter–level control) and a depth–wise in–plane rotation (intra–level mixing). The transformed vector is then mapped back to the manifold. Concretely,

$$
\begin{aligned}
v_i &= \log_0^c(x_i) \in T_0\mathbb{B}^2, \\
\tilde{v}_i &= k_{d_i}\, R(\theta_{d_i}) \otimes v_i, \quad R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \\
x_i' &= \exp_0^c(\tilde{v}_i).
\end{aligned}
$$

Here $k_d \in \mathbb{R}$ and $\theta_d \in \mathbb{R}$ are learned per–depth parameters, initialised as k_d=1 and θ_d=0.

Intuitively, the scale factor k_{d_i} contracts or expands the Hyperbolic radius of points at depth d_i, aligning their radial placement with temporal level. The rotation $R(\theta_{d_i})$ re-arranges points within the same level, enabling intra–level mixing of sibling nodes. This ensures that the model can be robust to misplacement of nodes during the procedural clustering being used to graph trees. [8]. As mentioned above and in Section 3.4.1, log maps and exp maps are again used to retain Hyperbolic geometry while applying Euclidean operations. Below shows a Pseudocode for Depth-dependent Linear Gate.

[7] Experimentally, I found that including this mixer helped training. When the mixer is not included, training seems to stagnate.

[8] See Appendix A, Algorithm 5 for the clustering approach, and Section 3.1 for context.

Algorithm 2 Depth–dependent Linear Gate (2D Poincaré ball)

Require: $x \in \mathbb{R}^{L\times 2}$ with $\|x_i\| < R_c$, depths $d \in \{0,\dots,D{-}1\}^L$, curvature $c > 0$

$R_c \leftarrow 1/\sqrt{c}, \quad \varepsilon \leftarrow 10^{-15}$

Parameters: `depth_scale` $\in \mathbb{R}^{D\times 1}$ (init 1), `depth_theta` $\in \mathbb{R}^{D\times 1}$ (init 0)

for $i = 1 \to L$ **do**

  $d_i \leftarrow \min(d_i, D{-}1)$

  $v_i \leftarrow \log_0^c(x_i) \in \mathbb{R}^2$

  $k \leftarrow$ `depth_scale`$[d_i, 0]$

  $\theta \leftarrow$ `depth_theta`$[d_i, 0]$

  $R(\theta) \leftarrow \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$

  $\tilde{v}_i \leftarrow k\,\big(R(\theta) \otimes v_i\big)$

  $x'_i \leftarrow \exp_0^c(\tilde{v}_i)$

  $x'_i \leftarrow \mathrm{ClampToBall}(x'_i, R_c)$

end for

return $x' = (x'_1, \dots, x'_L) \in \mathbb{R}^{L\times 2}$

The resulting output from this channel leaves us with 1 embedding for each demonstration-frame, current observation and action given. It is important to note that the embeddings here are constructed from just actions derived from observations from demonstration and the current observation, and point clouds are not used here.

3.6 Product Manifolds (Mixed Curvature)

[16, 18] presents an interesting idea of embedding within a mixed-curvature embedding space. Intuitively, mixed-curvature space can be thought of as a product of multiple Spherical, Hyperbolic and/or Euclidean spaces.

Formally, [16] call this space the Product Manifold $\mathcal{P}$, and it is defined to be the Cartesian Product of the a sequence of Manifolds $\{\mathcal{M}_\rangle\}_{i=1}^k$ from different geometric space.

$$\mathcal{P} := \mathcal{M}_\infty \times \mathcal{M}_\in \ldots \times \mathcal{M}_\| \tag{3.13}$$

With this definition, points $p \in \mathcal{P}$ can simply be expressed as a concatenation of points from the constituting manifolds. With this, the tangent vector of product Manifold $v \in T_p\mathcal{P}$ will also

be written similary.

$$p = (p_1, p_2 ... p_k), \text{where } p_i \in \mathcal{M}_\rangle$$
$$v = (v_1, v_2 ... v_k), \text{where } v_i \in T_{p_i}\mathcal{M}_\rangle$$

This then allow us to decompose exponential map and logairthm map (see Section 3.4.1) of a product manifold into its individual components:

$$exp_p(v) = ((exp_{p_1}(v_1) ... exp_{p_k}(v_k))$$
$$log_v(p) = ((log_{v_1}(p_1) ... log_{v_k}(p_k))$$

This also implies that distances decompose additively across components, which underpins our later attention fusion [16]:

$$d_P^2\big((p^E, p^H), (q^E, q^H)\big) = d_E^2(p^E, q^E) + d_H^2(p^H, q^H). \tag{3.14}$$

4 Methodology

Contents

4.1 General Overview of Model

In this paper, I propose a Hyperbolic variation of Instant Policy. Much like in Instant Policy, I have framed the problem of ICIL in a diffusion-based graph generation setting, whereby my model learns to predict per-node denoising directions that can be used to derive denoising directions for

noisy actions. The model will be working in a 2D environment, where observations consist of agent keypoints in world frame, segmented 2D point clouds and agent state. The action a_t taken by the agent at time t can be defined by a 3-tuple (translation, rotation, state-change), where state-change $\in \{0, 1\}$. Demonstrations are expressed as sequences of observations $\{d_{ij}\}_{i,j=1}^{N,L}$, N being the number of demonstration and L being the demonstration length. The model will take in noisy actions $\tilde{a}_t$, demonstrations and the current observation o_t, and output per-node denoising direction that is used to denoise the noisy action. Figure 4.1 shows a brief overview of my model. For a more detailed figure, see Figure E, which I have left in Appendix E due to its size.

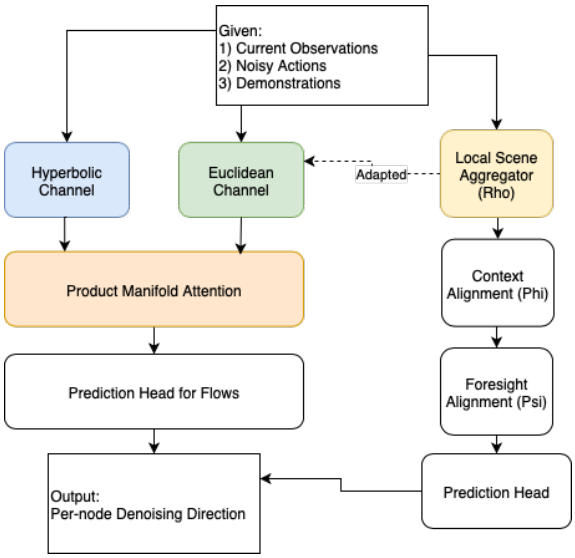


Figure 4.1: Agent Architecture (Brief). Figure showing a simplified overview of Instant Policy and my proposed method. Dotted lines indicate which parts I have adapted. Rho, Phi and Psi are each individual components of Instant policy, with their roles labelled.

Abstractly, the model has 2 channels of information : (1) Euclidean embeddings, which is responsible for extracting and propagating semantic information within spatial features and (2) Hyperbolic Embeddings, which shares the same responsibilities but for semantic information within actions. The aggregated features from both channels are then passed through Product Manifold Attention layer which aims to align demonstration embeddings with the current and post-action state[1] embedding (for both Hyperbolic and Euclidean space) in a Product Manifold [16]. The output of this layer gives us the current and post-action state latent variables. Like in instant policy, we propagate information from the current latent variables to each post-action state latent variables, and use them to predict per-node denoising directions. These denoising directions can then be used to derive actions as per [1].

[1]What I mean by post-action state is much in similar spirit as [1], see in Section 3.2.1

4.1.1 Why 2 channels?

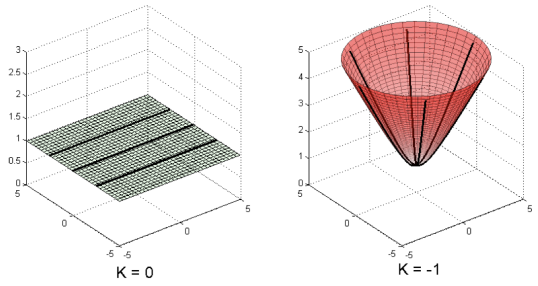


Figure 4.2: Euclidean space vs Hyperbolic Space. K in this case is curvature, and the bold lines being Geodesics. Notice that at zero-curvature, geodesics remain equidistant, but at negative-curvature, it grows far-apart exponentially. This underpins the exponential-volume growth in Hyperbolic space. Taken from [16]

When studying Instant Policy, it became apparent that requiring a single embedding space to capture both local geometrical relations and global task semantics is limiting. The Euclidean channel is well suited to modelling local spatial structure and linear transitions, which makes it effective for capturing object geometry and agent-environment interactions[2]. However, from Section 3.1, I argued that the action-semantic structure is more naturally hierarchical and tree-like, resembling a knowledge graph [17]. Such structures are more effectively embedded in Hyperbolic space, where exponential volume growth provides a natural inductive bias for representing hierarchies (see Fig 4.2).

This motivates the use of a dual-channel architecture in which Euclidean embeddings encode local spatial geometry while Hyperbolic embeddings encode global semantic structure. To exploit the complementary strengths of these two spaces, we integrate them using a product manifold, which preserves their geometric properties while enabling joint reasoning across modalities [16].

The relative nature of actions and observations further supports this design. Actions are always taken with respect to observations, and their relationships can vary depending on the task. For instance, tasks such as balancing often exhibit oscillatory, near-cyclical patterns between actions and observations. While spherical manifolds may be more appropriate for modelling such cyclicity [16], incorporating a spherical channel was beyond the scope of this work. Nevertheless, the product manifold framework provides the flexibility to extend in that direction, and in principle point clouds could be encoded within a spherical space to capture cyclical distributions in confined

[2]Instant Policy has already demonstrated that ICIL can be achieved using Euclidean embeddings alone.

robot workspaces.

In summary, a dual-channel design allows each embedding space to focus on the aspects of the data it represents most effectively – Euclidean for spatial grounding and Hyperbolic for semantic hierarchy – while product manifold attention provides a principled means of integrating them for robust in-context imitation learning.

4.2 Hyperbolic Embedding channel

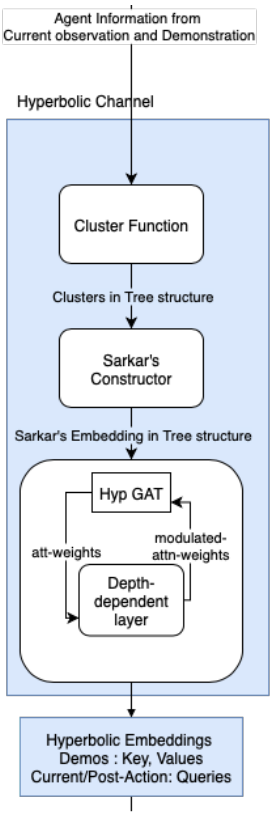


Figure 4.3: Hyperbolic embedding channel

We operationalise the preliminaries from Chapter 3 by building hierarchical trees from orientation/state transitions (see Section 3.1) and embedding their nodes in the Poincaré ball via Sarkar's constructor (Section 3.3). Temporal awareness is injected with a Hyperbolic GAT equipped with a depth-dependent gate (Section 3.5.1). This yields one Hyperbolic embedding per demo frame, plus embeddings for the current observation and post-action state.

4.2.1 Purpose

The Hyperbolic embeddings have 2 purposes. Individually, each embedding needs to reflect the semantic state from which that action is taken from, and as a whole, they serve as a **signature** for the entire action trajectory, with respect to the current context defined by demonstrations.

To elaborate on the first part, in Section 3.1, I propose that semantic shifts within actions can be effectively captured by Hyperbolic embeddings [3]. Extending from this idea, we now propose that by reflecting semantic transitions of human demonstrations, Hyperbolic embeddings constructed from human demonstration should be able to capture the semantic state from which each action is taken.

Moving on, we emphasise that the Hyperbolic embeddings for each demonstration are constructed with respect to the current observation. This construction inherently makes the embeddings task-local – they are valid only within the specific task under consideration. Far from being a drawback, this task-specificity is desirable in the ICIL setting. The aim of ICIL is not to build universal embeddings that generalise across unrelated domains, but rather to capture the semantic and structural regularities of the demonstrations that define the current task. In this way, the embeddings remain tightly aligned with the problem the agent is solving.

Since the topology of the graph is determined by the action trajectory, Hyperbolic embeddings provide a natural means of encoding this structure. Each generic type of trajectory—that is, a broad solution strategy for completing the task—can therefore be associated with a distinct signature in the Hyperbolic space. Although infinitely many fine-grained trajectories could achieve the same outcome, angular-granularity based clustering groups these trajectories into tree structures, such that semantically similar solutions are represented similarly. This clustering increases the robustness of the agent to variation across demonstrations while preserving the distinctions between qualitatively different solution strategies.

Accordingly, each demonstration can be assigned a unique, task-specific embedding signature. By operating over these signatures, the agent avoids distraction from irrelevant variations and instead focuses on representations that are valid within the current task context. The product manifold attention mechanism then facilitates selection among these trajectory signatures, allowing the agent to align with the most appropriate solution strategy during inference and, in turn, achieve stronger task performance.

[3] albeit naively

4.2.2 Clustering for Trees

While Appendix A, Algorithm 5 outlines the pseudo-code for recursively clustering actions (as observation transitions), I will like to give some explanation in relation to Section 3.1.

In Section 3.1, I have mentioned how I see actions as being something that can be recursively divided into its constituent components. With this mindset in mind, if I were to recursively cluster actions based on granularity of these constituent components, I should end up with a good approximation for the hierarchy relations within the action. However, this idea is admittedly naive, and there is no guarantee that the resulting clusters will reflect the underlying hierarchy. [4] mentions this as an inherent difficulty when utilising a systematic procedure to define hierarchies. It is typically hard to define such procedures such that the actual underlying hierarchy is reflected. There is a trade-off between fineness-coarseness and meaningful-meaningless. Going 'too fine' may just incur unnecessary computation and risk convergence in clustering results, while going 'too coarse' risk resulting in meaningless clusters.

However, given the simplicity of movement in a 2D setting, and having a general expectation on the intricateness of tasks that the model is expected to perform, it is possible to come up with a reasonable set of granularities that is capable of approximating the underlying tree-structures. Furthermore, in Section 3.5.1, I explicitly utilised a depth-dependent linear gate to modulate attention weights within the constructed tree, so that the model remains robust to such misclassifications by the clustering algorithm (see Section 3.5.1). Lastly, although the clustering procedure is naive, it gives me great control over the depth of the Hyperbolic tree. This is done by defining number of angular granularities, which sets the max depth for any given leaf node. This avoids the numerical hurdles mentioned by [6] near ball boundary.[4]

4.2.3 Linking current and action embeddings with each demonstration's Sarkar embeddings

We now describe how the current and action embeddings are linked to the demonstration trees instantiated above. Sarkar's embeddings are constructed from a tree, meaning that for any node to receive a meaningful embedding, it has to exist somewhere within the tree. We have only mentioned how we can reduce demonstrations into Sarkar's embeddings, but have not talked about how we

[4] Personally I am not against this simplicity. Since the application of Hyperbolic methods into Instant Policy is in its very early stage, simplifying things gives me a better understanding of what works and what does not work without overwhelming complexities.

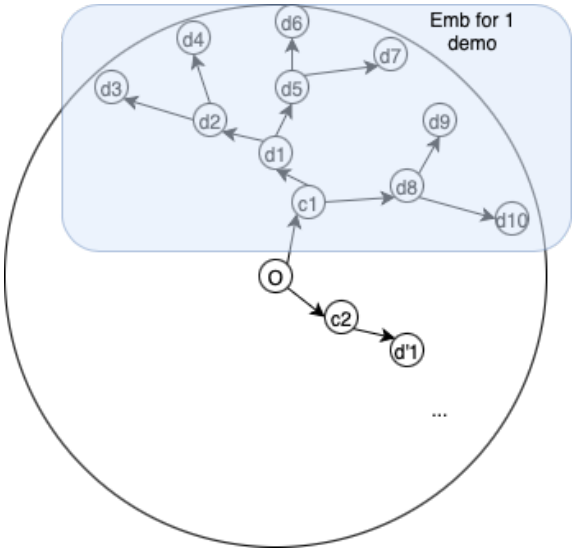


Figure 4.4: Figure shows an example of how embeddings will look in a Poincaré ball model. Note the for each demonstration D_i (where $i \in [1 : num\_demos], D_i = \{d_j\}_{j=1}^{demo\_len}$), an embedding is constructed for the current observation c_i.

are going to utilise them to derive Sarkar's embeddings. While some sort of learning procedure like Euclidean Channel alignment can be used to learn the current Hyperbolic embeddings[5], there is actually a more computationally affordable way that we can retrieve a meaningful Hyperbolic embeddings for the current Hyperbolic embedding. To do this, before embedding the demonstrations, we put the world frame's origin and our current observation before each of our demonstration. We do not consider putting current observation as root since it will result in the current Hyperbolic embedding being 0 by definition and this can cause gradient problems[6]. For simplicity, I kept the post-action state embeddings the same as current Hyperbolic embeddings, but the same idea can be applied by putting these post-action state demonstrations after the current observation before clustering to construct tree.

This construction is loosely motivated by the framework of Coarse-to-Fine Imitation Learning [40], where the key insight is that if an agent can align itself with the state at which a demonstration begins, it can then simply replay the demonstration to reproduce the task. By inserting the current observation before each demonstration in the tree, we are enforcing an analogous structure: the model learns to anchor demonstrations relative to the agent's present state, such that reproducing the demonstration amounts to replaying the subsequent trajectory from that anchored point. Fig 4.4 shows a visual of how this will look.

[5]This was my initial Idea. However, on further analysis, this does not work as well as the Euclidean channel is temporally-agnostic, while the Hyperbolic channel is temporally-aware.

[6]Initially I placed current as root, and the model struggled with learning significantly

4.3 Euclidean Embedding channel

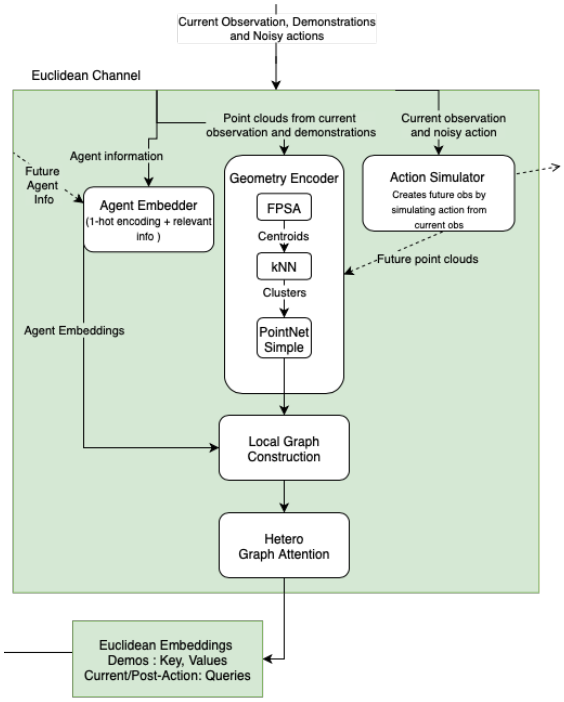


Figure 4.5: Euclidean embedding channel

We largely follow Instant Policy's Euclidean processing pipeline (see Chapter 3, Section 3.2) and adapt it to our setting with a simplified encoding step (see Appendix B).

4.3.1 Purpose

The purpose of the Euclidean embedding is to capture spatial information of the scene that it is produced from. They serve as a frame of reference for the agent when using Hyperbolic embeddings, which captures semantic information within its embeddings. Intuitively, with Hyperbolic embeddings, the agent can visualise the expected action trajectory, but it is unable to recognise the spatial scene the action was taken from. Euclidean embeddings serve to fill this gap, so that the agent is able to better localise itself. Intuitively, with only the Hyperbolic embeddings, the agent can have a general expectation on the action trajectory that it should take, but not from which physical state. Euclidean embeddings serve as the anchor that ties every action to their corresponding physical state, so that the agent is better able to infer the tasks at hand from the

demonstration[7].

4.3.2 Embedding Spatial Features

In Section 3.2.1, I mention that in Instant Policy, point clouds are processed via a SetAbstraction Layer to encode local geometry. Formally, this layer implements a 2-layer PointNet++ module, which can be abstracted as a 3 step process of sampling, clustering and encoding [31]. We retain this 3 step process, but substitute the final encoding step with a simpler variant. The rationale for this choice is detailed in Appendix B

Given a point cloud $P \in \mathbb{R}^{N\times 3}$, a predefined number of object nodes M, and neighbourhood size k for kNN clustering, our set abstraction layer operates as described below:

Algorithm 3 Set Abstraction Layer with Simplified Encoding

Input: point cloud $P \in \mathbb{R}^{N\times 3}$, number of centroids M, neighbourhood size k
Output: encoded features $F \in \mathbb{R}^{M\times d}$

Step 1: Sampling
$C \leftarrow \text{FPSA}(P, M)$ ▷ Select M centroids using Furthest Point Sampling

Step 2: Clustering
$\mathcal{N}(c_i) \leftarrow \text{kNN}(P, c_i, k)$, for each $c_i \in C$ ▷ Form clusters of size k around each centroid

Step 3: Relative Coordinates
$\tilde{\mathcal{N}}(c_i) \leftarrow \{p - c_i \mid p \in \mathcal{N}(c_i)\}$ ▷ Express cluster members w.r.t. centroid

Step 4: Local Encoding
$f_i \leftarrow \text{PointNetSimple}(\tilde{\mathcal{N}}(c_i))$, for each $c_i \in C$
$F \leftarrow \{f_1, f_2, \ldots, f_M\}$

For brevity and completeness, pseudo-code for FPSA can be found in Appendix A, Algorithms 4.

4.3.3 Parsing Information into Agent Nodes

After point cloud processing, we obtain M centroids with features $\mathcal{F}_i$, which serve as object nodes. Agent node embeddings are defined by concatenating a one-hot identifier with agent orientation $(sin(\theta), cos(\theta))$, agent state, time, and the *done* flag (indicating task termination), followed by a linear projection. As in Instant Policy [1], edges are placed between every agent node and object node, and embedded using Eq. (3.1), yielding the *Local Graph*. Finally, the hetero-graph attention layer from Eq. (3.3) transforms the *Local Graph*, allowing spatial information to be aggregated into the agent node embeddings. From this Euclidean channel, we get 1 embedding per agent-node.

[7] or more of infer why Y action is taken at X state, since we are doing ICIL

This is done for all observations : from demonstrations, from the current observations and the post-state observations. However, unlike in Instant Policy, where an additional Hetero-layer (3.3) is applied on demonstration embeddings and current embeddings, we deviate from Instant Policy from this point onwards. Embeddings retrieved from our channel will be temporally-agnostic, and they seek to only embed the spatial information at that given time.[8]

4

4.4 Bridging Euclidean and Hyperbolic Channels

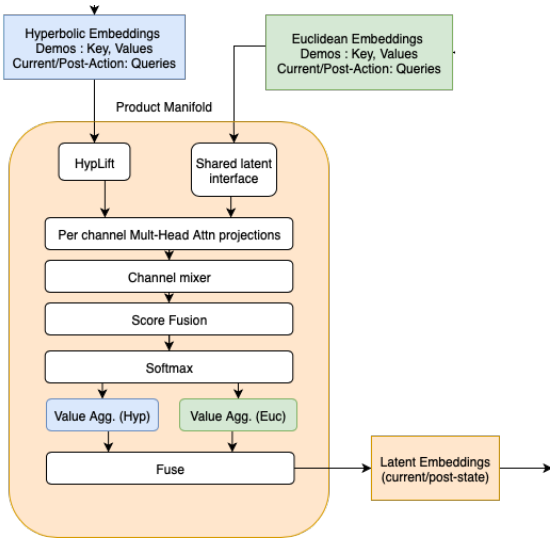


Figure 4.6: Product Manifold Attention

4.4.1 Purpose

Now that the agent has 2 channels of information coming in, it needs a way to combine them such that (1) it leads to meaningful representation, (2) demonstrations are utilised effectively. Effectively, the final latent variable produced at this point should be able to reflect the nature of the task at hand and the desired action to take. To achieve this, a Product manifold with attention is utilised, such that (1) is preserved via geometry-preserving mechanism[9], and (2) is achieved by having an attention mechanism using embeddings from demonstrations.

[8] This is done partly to differentiate my method from Instant Policy. I wanted to show that the Hyperbolic Channel can provide a better context alignment than what its done in Instant Policy. Including this will not make my argument strong.

[9] Distorting geometry will make said embedding meaningless to its counterparts, since it is no longer valid

4.4.2 The bridge

Let $x_i^E \in \mathbb{R}^{d_E}$ and $x_i^H \in \mathbb{B}_c^d$ denote the Euclidean and Hyperbolic embeddings for item i (agent node, demo frame, or action state). We expose a *shared latent interface* by mapping both channels to a common z-dimensional space:

$$\phi_i^E \;=\; W_E\, x_i^E \;\in\; \mathbb{R}^z, \qquad \phi_i^H \;=\; W_H\, \tau(x_i^H) \;\in\; \mathbb{R}^z, \tag{4.1}$$

where $W_E \in \mathbb{R}^{z\times d_E}$ and $W_H \in \mathbb{R}^{z\times d}$ are learnable, and $\tau(\cdot)$ is a tangent-space lift for Hyperbolic points (we use the log map at the origin, $\tau(x) = \log_0(x)$, with a learnt per-channel scale s_H applied before the linear layer to balance norms).[10] We also include lightweight normalisers $\hat{\phi}_i^E = \mathrm{LN}(\phi_i^E)$ and $\hat{\phi}_i^H = \mathrm{LN}(\phi_i^H)$ to align second-order statistics before attention.

Given a current state i=cur and a set $\mathcal{D}$ of demo items (frames or nodes), we form *per-channel* queries, keys and values:

$$Q^E = A_Q^E \hat{\phi}_{\mathrm{cur}}^E,\; K^E = A_K^E \hat{\phi}_j^E,\; V^E = A_V^E \hat{\phi}_j^E \quad \text{and} \quad Q^H = A_Q^H \hat{\phi}_{\mathrm{cur}}^H,\; K^H = A_K^H \hat{\phi}_j^H,\; V^H = A_V^H \hat{\phi}_j^H,$$

with $A_\bullet^{(\cdot)} \in \mathbb{R}^{h\times(z\times z/h)}$ being the usual multi-head projections. This 'interface' is the hinge that lets us combine channels cleanly while preserving each geometry upstream.

4.4.3 Geometry-aware score fusion.

We compute Euclidean and (lifted) Hyperbolic attention logits and fuse them with learnt non-negative weights:

$$s_{ij}^E = \frac{\langle Q_i^E, K_j^E\rangle}{\sqrt{d_h}}, \qquad s_{ij}^H \;=\; \frac{\langle Q_i^H, K_j^H\rangle}{\sqrt{d_h}}, \tag{4.2}$$

$$\lambda^E, \lambda^H = \mathrm{softmax}\big([\gamma^E, \gamma^H]\big) \quad \Rightarrow \quad s_{ij}^P \;=\; \lambda^E\, s_{ij}^E + \lambda^H\, s_{ij}^H. \tag{4.3}$$

Here d_h=z/h, and (γ^E, γ^H) are learnt *channel logits* (either global, per-head, or conditioned on i via a small MLP). The product-space attention weights are then

$$\alpha_{ij}^P = \mathrm{softmax}_j(s_{ij}^P), \qquad Z_i^E = \sum_{j\in\mathcal{D}} \alpha_{ij}^P\, V_j^E, \quad Z_i^H = \sum_{j\in\mathcal{D}} \alpha_{ij}^P\, V_j^H. \tag{4.4}$$

[10] Other lifts are possible; I found $\log_0$ stable and sufficient in practice.

Finally we fuse the per-channel outputs:

$$z_i \;=\; W_F\,[\,Z_i^E\,;\,Z_i^H\,]\;+\;b_F \quad\in\quad \mathbb{R}^z, \tag{4.5}$$

yielding a *product-manifold latent* that conditions the policy heads. This construction mirrors the decomposability of product-manifold distances Section 3.6 while keeping attention operations in stable vector spaces (tangent/Euclidean).

4

4.4.4 Attention within Product Manifold

This instantiates the mixed-curvature product (cf. Chapter 3, Section 3.6) with a geometry-aware attention operator. Building on the *shared latent interface* from Section 4.4, Product Manifold Attention (PMA) realises a single attention operator whose scores blend Euclidean and (lifted) Hyperbolic similarities[16].

We combine the two channels via a PMA layer that aligns the current state with demonstration items across both geometries. In line with mixed-curvature products [16], our design respects the additivity of component distances.

Inputs and projections : Given Euclidean embeddings x^E and Hyperbolic embeddings x^H (Section 4.4), PMA first lifts x^H to the tangent space and projects both channels to a shared latent dimension ($\phi^E, \phi^H \in \mathbb{R}^z$). Per-channel query/key/value tensors are constructed with standard multi-head projections.

Score fusion in product space : PMA computes per-channel logits s^E, s^H and blends them with learnt mixture weights (λ^E, λ^H) to yield a *product* score $s^P = \lambda^E s^E + \lambda^H s^H$. The weights are conditioned on the current query (small MLP on $[\hat{\phi}^E_{\text{cur}}; \hat{\phi}^H_{\text{cur}}]$), enabling the model to favour Euclidean cues for precise spatial alignment and Hyperbolic cues for hierarchical/semantic alignment[16].

Value aggregation and fusion : A single set of product-space weights α^P aggregates the two value streams: $Z^E = \sum_j \alpha^P_{\cdot j} V^E_j$ and $Z^H = \sum_j \alpha^P_{\cdot j} V^H_j$. The outputs are concatenated and linearly fused, producing the *current* and *action* latents used downstream by the policy to predict per-node denoising directions.

Why this should work : (i) *Complementary inductive biases*: Euclidean features capture local metric structure; Hyperbolic features encode hierarchical semantics. (ii) *Mixed-curvature fidelity*: by emulating the additive structure of product-manifold distances with an additive score, PMA reduces the distortion incurred by forcing everything into a single space. (iii) *Learned gating*: the mixture (λ^E, λ^H) lets the model adaptively privilege the geometry that best explains the present context, improving alignment between current observations and demonstrations.

4.5 Towards Action

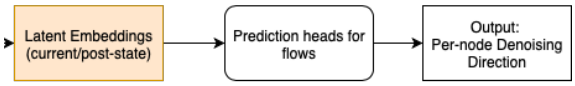


Figure 4.7: Predictions to Flows

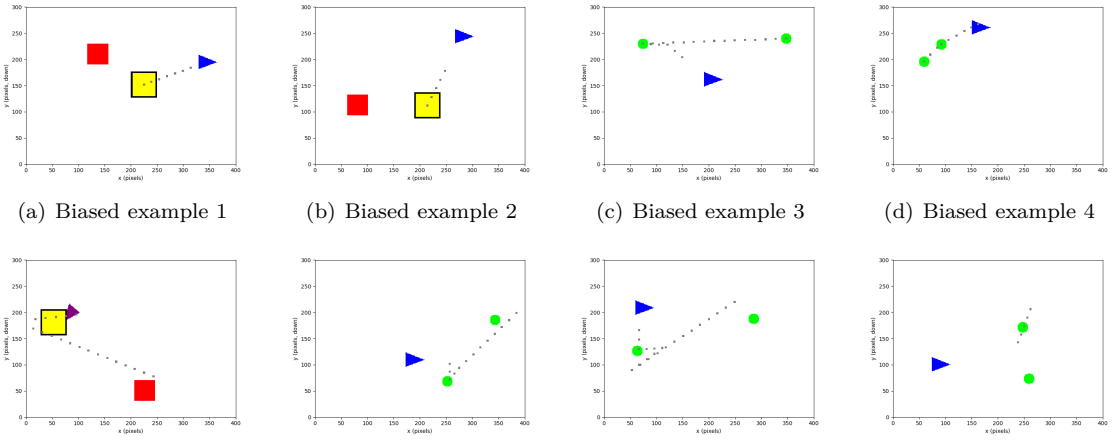
Finally, like in Instant Policy, the latent variables are utilised to predict **Flows**. Flows in our 2D case consist of $(t_x, t_y, r_x, r_y, s_g)$ where t_k represents the shift in k-axis caused by translation, and r_k represents shift in k-axis caused by rotation.

4.6 Training

For training, we utilise pseudo-demonstrations as in [1]. Each batch contains (i) a set of procedurally generated demo trajectories (random and biased variants), (ii) the current observation graph, and (iii) noisy actions $\tilde{a}_t$. The Euclidean and Hyperbolic channels are computed independently, then fused by PMA to produce current/action latents. The denoising head predicts per-node directions; losses are standard ℓ_2 denoising objectives on action nodes, with auxiliary alignment losses that encourage PMA to place higher mass on demo frames consistent with the current state. We apply teacher forcing on diffusion timesteps and follow the noise schedule of [1, 41].

4.6.1 Pseudo Demonstrations

Pseudo Demonstrations are demonstrations that are constructed through a defined systematic process. There are 2 types of pseudo demonstrations. The first type of pseudo demonstrations is **Random**, where waypoints are randomly selected to be either on the object or around the



(a) Biased example 1 (b) Biased example 2 (c) Biased example 3 (d) Biased example 4

(e) Random example 1 (f) Random example 2 (g) Random example 3 (h) Random example 4

Figure 4.8: Pseudo demonstration examples. Green objects represents edible, yellow goals and red obstacles. Agent is the triangle, and its state is determined by its colour : Blue (off), Red (on). Grey dots depicts waypoints that agent will traverse.

object. The second type of pseudo demonstrations is **Biased** where we pick waypoints that mirrors tasks that agent expects to solve. The second type of demonstrations are utilised to urge agent to learn to solve tasks. These pseudo demonstration do not seek to accurately simulate expert demonstrations. They do not account for complex collision physics, meaning that they are capable of planning waypoints through object. For example, in Instant Policy, pseudo demonstrations do not even simulate the complex contact physics when gripping objects, instead gripping is simulated by sticking the object to the agent when the agent performs grip near or on the object.

While this pseudo demonstration may seem to lack significant semantic that will allow the agent to perform tasks, it is important to remember that we are performing ICIL. In ICIL, the aim of the agent is not to learn to perform the tasks, but to perform a task given demonstrations. When using pseudo demonstrations, we do not want the agent to learn policy tied to tasks defined by pseudo demonstration. We aim for the agent to learn to extract the underlying semantics from these semantically-consistent pseudo-demonstrations, and from there, learn to predict action for the current training task [11]. The reason why this is possible is that since pseudo demonstrations are procedurally generated, they do have an underlying semantic that is consistent throughout all pseudo demonstrations. For the case of **Random**, the semantic is simply *"go near the object and interact with it"*. Experimentally, Instant Policy [1] have shown that their model outperforms state-of-the-art methods (BC-Z, Vid2Robot, GPT2) in simple tasks (Open box, Lift Lid ect) using only pseudo demonstrations for training, while the state-of-the-art methods used a combination of pseudo demonstrations and human demonstrations for training. However, it is also wise to

[11] Note that the training task is generated as a pseudo demonstration too.

note that these methods might not have adapted themselves for the use of pseudo demonstrations. Regardless, the fact that [1] is able to perform these tasks at 0.71 success rate using only pseudo demonstrations goes to show that they can be a powerful source of data. If anything, when augmented with Human Demonstrations, success rate for instant policy on these tasks increased to 0.97.

In summary, the idea of consistent semantic within generated pseudo demonstration, and the ability of the agent to (1) extract these semantic from given demonstration and (2) utilise them to make action prediction is what make these pseudo demonstration effective data for training. Figure 4.8 shows some examples of pseudo demonstrations.

4.7 Deployment

At deployment time, the agent receives one or more human demonstrations and a current observation. Noisy action trajectories are first sampled from a standard Gaussian prior. These latent actions are iteratively denoised using a deterministic DDIM scheduler, following the procedure in [1].

At each denoising step, the current state and demonstrations are embedded via the Euclidean and Hyperbolic channels, fused through the PMA layer, and passed into the policy head to predict per-node denoising directions. These directions specify how to refine the noisy action nodes towards clean actions. To ensure that action predictions respect the geometry of demonstrations, we align predicted denoising vectors with demonstration embeddings using a singular value decomposition (SVD) step, exactly as in Instant Policy. This alignment stabilises training-free deployment and improves trajectory consistency.

After the final denoising step, the clean actions are extracted from the converged action nodes. The agent can then execute them directly in the environment. In practice, this should produce smooth and semantically consistent action sequences that closely follow the structure implied by the demonstrations, while preserving robustness to noise in the sampled latent actions.

4

5 Experimentation

Contents

5.1 Baseline Performance Comparison with Instant Policy

In this section I would like to first baseline my model's performance against Instant Policy. Then I will analyse core assumption within my model experimentally to explain why it has failed.

5.1.1 Experiment Setup

Training Parameters : Models were trained with the following hyper-parameters:

Learning rate	1×10^{-6}	Number of demonstrations	2
Hyperbolic dimension	2	Demonstration length	20
Number of attention heads	8	Maximum diffusion steps	1000
Euclidean head dimension	32	Maximum flow translation	100
Latent variable dimension	$8 \times (2 + 32)$	Maximum flow rotation	100
Number of sampled point clouds	8	Biased ratio	0.5
Curvature	1 (Poincaré ball)	Num optimisation steps	50000

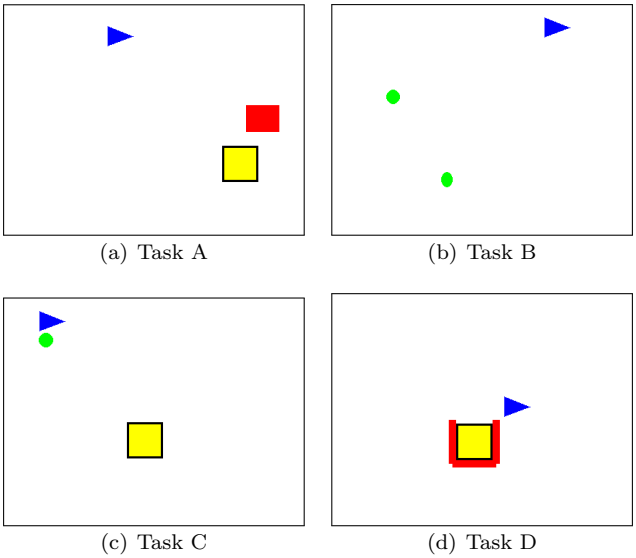
The loss function utilised is mean squared error, defined as `nn.MSELoss(reduction='sum')`. Agents are trained only on pseudo-demonstrations that imitate a subset of the evaluation tasks (task A and B, see table 5.1.1. These pseudo-demonstrations are continuously generated as the agent trains. This is more robust than storing a set of pseudo-demonstrations as we run the risk of model optimising for the pseudo-demonstrations if the dataset is not large enough[1].

Evaluation Protocol : During evaluation, each model denoised randomly sampled actions over 1000 steps and was tested on four task types of increasing difficulty (10 games per task):

1. **Reach Goal**: shortest and most direct task. Semantic can be found in PseudoDemos. Average Demo Length[2] (ADL) = 17.7, Min length = 10, Max length = 29.
2. **Eat All Edibles (2 objects)**: longer trajectories than goal reaching. Semantic can be found in PseudoDemos. ADL = 30, min length = 17, max length = 45
3. **Push & Place**: requires state-awareness to push an object to a target location. Semantic not found in PseudoDemos. ADL = 50.4, min length = 20, max length = 61
4. **Parking**: highly intricate; demonstrations vary widely, requiring generalisation across diverse strategies and fine-grained control. Semantic not found in PseudoDemos. ADL = 57.2, min length = 21, max length = 80

The first two tasks are relatively straightforward, whereas *Push & Place* and *Parking* are substantially more challenging. In particular, the *Parking* task is expected to exceed the current models' capabilities due to its complexity and the need for precise, fine-scale actions. Figure 5.1 shows examples of each class

[1] Diffusion Models,GAT,GNNs are generally strong learners
[2] Calculated from 10 demos, rounded down



(a) Task A (b) Task B

(c) Task C (d) Task D

Figure 5.1: Examples of each task

5.1.2 Results

The comparative results are summarised in Table 5.1. The proposed model did not surpass the Instant Policy baseline and the overall performance was not comparable. The model was only able to achieve a very limited degree of ICIL. As expected, performance varied considerably across the different task types.

Table 5.1: Model performance comparison between the proposed model and Instant Policy.

Task	**Proposed Model**	**Instant Policy**
A: Reach Goal	2/10	5/10
B: Eat All Edibles	1/10	3/10
C: Push & Place	0/10	0/10
D: Parking	0/10	0/10

I would like to note that in Task B, although both models usually fail, it often manages to get 1 edible, meaning that it did, to some extent, learn from demonstration but just not to its completion.

5.1.3 Discussion

Firstly, I would like to address the difference in performance in Instant Policy as compared to performance cited in [1]. The only process I omitted when implementing Instant Policy was its demonstration processing, where one explicitly selects 'good' demonstration-frames when down-sampling demonstrations. Instead, I opted to downsample 'evenly', as I thought that tasks were

relatively simple, and there was not much difference in significance between demonstration frames. However, this was a mistake on my part and as a result, during evaluation the policy is prone to oscillate, making its performance inconsistent[3].

I also suspect that the geometry encoder that I am using is not as good as Pointnet++. Since that was the only major change in agent architecture when I was implementing Instant Policy, I believe that this must have played a role in the dip in performance. However, the geometry encoder's loss did converge towards 0.03+ (trained like occupancy net in Instant Policy, with pseudo-demo data), which suggests great accuracy, and it is trained on pseudogame objects, meaning it should generalise. My conclusion is that either the model over-fitted, or that the embeddings produced are not meaningful enough. I should have been more careful and critical of this component, and should have utilised a more established 2D geometric encoder to replace PointNet++ (see Appendix B for more information).

Moving on, I would like to elaborate on the disparity in performance across tasks. One likely explanation for the disparity between tasks lies in the quality of demonstrations. As noted in [1], demonstration quality plays a critical role in model performance. Because demonstrations were downsampled to fit a fixed maximum length, longer tasks were disproportionately affected. Tasks C and D typically require demonstrations of 40 or more steps, while Task B averages around 30. In contrast, Task A has an average demonstration length close to around 20, and both models therefore performed relatively well on this task.[4]

Another contributing factor is the use of biased pseudo-demonstrations. In this work, the pseudo-demonstrations were designed primarily to imitate Tasks A and B, which may explain why both models achieved better outcomes on these tasks. The authors of [1] similarly reported improved performance when biased pseudo-demonstrations were incorporated into training, reinforcing the importance of this strategy.

Finally, the failure of both models on Task D (Parking) is consistent with the inherent difficulty of the task. Parking requires precise, fine-grained adjustments in orientation and position, which are easily lost through downsampling. In the case of the proposed model, the use of relatively coarse angular granularity bins may further obscure these subtle semantic transitions. Since much of the task's semantic information is concentrated within fine manoeuvres (e.g., aligning the agent

[3]For task A, out of 10, I sometimes can get anywhere between 0-5/10 tasks succeeded. Likewise for task B, where I got between 0-3/10 tasks succeeded. I just took the best performance as they are more reflective of the actual Instant Policy, and as such provided a stronger baseline.

[4]I have limited the length of demonstrations for computation efficiency. Processing all 'long' demos will take a lot of time during training and evaluation.

precisely within a confined space), it is plausible that these details were underrepresented in the demonstrations, leading to poor generalisation.

5.2 Action Semantic: Did it work?

5.2.1 Motivation

In an attempt to understand why my method did not work as well compared to Instant Policy, I first investigate my assumptions on Hyperbolic embeddings. The Hyperbolic embeddings constructed through the demo handler are designed to reflect sequential structure in actions, not just isolated states. Intuitively, if two trajectories follow different paths through the task space (e.g., up–right vs. left–down), their embeddings should also be separable in latent space. In other words, the embeddings should carry sufficient information about the temporal sequence of actions that generated them.

The purpose of this experiment is therefore to test whether the frozen demo handler produces embeddings that retain enough fine-grained information such that the next action token can be *linearly* decoded from them. A linear probe provides a minimal-capacity diagnostic of linear separability without allowing the probe to learn the task.

5.2.2 Experiment Setup

Encoder: A frozen demo handler produces per-step Hyperbolic embeddings which are mapped to the tangent space via log_0

Labels: Next-token y_{t+1} from a 12-class vocabulary (E,NE,N,NW,W,SW,S,SE,ROT+,ROT−,STOP,[EOS]).

Splitting: Disjoint train/validation/test by base trajectory ID to prevent leakage.

Probes and baselines:

(i) Linear probe (softmax over an affine map of embeddings) — the main test.

(ii) Token-only baselines (Last-token, Bigram) — sanity checks for short-range token dependencies.

(iii) A small MLP head (2×128) on embeddings — tests non-linear separability.

Scale: 5000 episodes used.

5.2.3 Results

Method	Val Acc	Test Acc
Last-token (baseline)	0.246	0.269
Bigram (baseline)	0.328	0.366
MLP (2×128) on embeddings	0.343	0.340
Linear probe on embeddings	0.188	0.185

Table 5.2: Next-token prediction accuracy (higher is better).

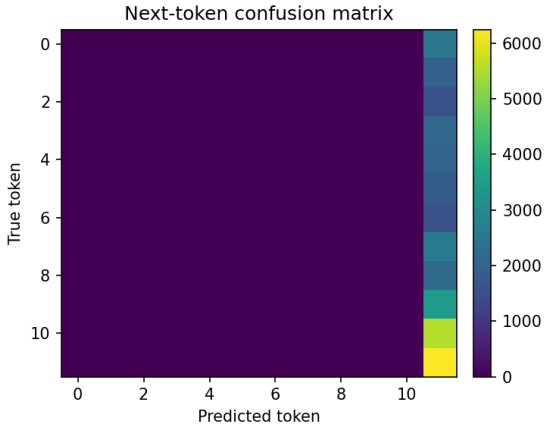


Figure 5.2: Confusion Matrix of embeddings. As seen, the most-predicted class by the probe is [EOS] (ID 11)

5.2.4 Discussion

Do the embeddings linearly encode next-action semantics? On this evidence, no. The linear probe achieves only **0.188** (val) and **0.185** (test), substantially below both token-only baselines (Bigram: **0.328**/**0.366**; Last-token: **0.246**/**0.269**). If the embeddings carried fine-grained temporal signal that was linearly accessible, the linear probe would close the gap to (or exceed) these token-only baselines.

Is there any non-linear signal? A small MLP improves to **0.343**/**0.340**, indicating some non-linear structure is present in the embeddings. However, this still does not surpass the Bigram test accuracy (**0.366**), suggesting that short-range token dependencies remain a stronger predictor of the next action than what is recoverable from the frozen embeddings.

Class bias and imbalance. The probe's most-predicted class is [EOS]. This points to either (i) class imbalance near sequence ends, (ii) calibration issues in the probe, or (iii) embedding invariances that collapse distinctions among local actions, biasing predictions toward a frequent "safe" class. This manifest as a bright final column in the confusion matrix in Fig 5.2.

5

6

Conclusions and Future Directions

The experiments have shown that my proposed model is very limited in its capability to perform ICIL, as my core assumption about local action semantics was incorrect: next–action information is not linearly encoded in the frozen Hyperbolic embeddings, and only weakly captured by a small non–linear head. In contrast, Instant Policy is trained end–to–end with a next–step denoising objective that directly models fine–grained local dynamics, which plausibly explains its superior performance.

The challenge of demonstration quality – already a known limitation in Instant Policy – remains unresolved here. Owing to time constraints, the model presented is an initial prototype rather than the full realisation of the original vision. The intended next step is to leverage the global role of the embeddings by (i) representing demonstrations as **tree–structured** signatures and (ii) employing a **low–to–high resolution** graph–generation model. This builds on the view that, for any given task, an underlying (sparse) distribution generates sequences of semantic waypoints of variable length, with demonstrations as samples. Tree–based representations, paired with hierarchical generation, offer a principled route to generalising semantic information while reducing sensitivity to demonstration variations.

Although only partially explored here, this roadmap suggests concrete avenues for improving ICIL robustness: strengthen local dynamics (Eg: lightweight sequence–predictive or contrastive auxiliaries during embedding construction) while exploiting global signatures (tree–structured embeddings + multi–resolution generation). Together, these steps directly target the gap exposed by the Action–Semantics experiment and the outstanding issue of demonstration quality.

6

Declarations

I declare the use of ChatGPT-5 (OpenAI, https://chatgpt.com) and Claude Sonnet 4 (Antrhopic, https://claude.ai) to:

1. rephrase my wording for clarity, objectivity and correctness
2. aid me with writing pseudo-codes in latex
3. find relevant research papers in topics like hyperbolic embeddings and ICIL
4. aid with me understanding research papers
5. give me advice on optimising my code and making it cleaner
6. give me boilerplate code

6.1 Github Repos link

I have done a couple of rounds of refactoring throughout the project, and since I have implemented 2 models, I have multiple repositories. Below are the list :

1. `https://github.com/BeanBois/instant_policy_pp` For submission, **stagnant**
2. `https://github.com/BeanBois/my_icl_agent_duchess` Indv repo for main model, **live**
3. `https://github.com/BeanBois/my_instant_policy` Indv repo for IP (refactored + optimised) **live**
4. `https://github.com/BeanBois/icl-fyp` First attempt, **discontinued**

I will recommend checking the 1st link since its for submission. I might work on the 2nd and 3rd link so they might contain changes.

A

Auxiliary Algorithms

A.1 FPSA

This algorithm is used to downsample a cloud of pointclouds into K centroids by recursively selecting the point that maximises sum of distance from current selected centroid. Its aim is to provide a good 'coverage' over the set of point clouds, such that each local region has a allocated centroid that will be used to encode local geometry within geometry encoder.

Algorithm 4 Furthest Point Sampling

Require: Point set $P = \{p_1, \ldots, p_N\}$ in a metric space with distance $d(\cdot, \cdot)$; target sample size $K \leq N$

Ensure: Indices $\mathcal{S}$ of sampled points ($|\mathcal{S}| = K$)

$\mathcal{S} \leftarrow \emptyset$
Choose any seed index $s \in \{1, \ldots, N\}$ (e.g., uniformly at random)
$\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}$
$\text{dist}[i] \leftarrow d(p_i, p_s)$ for all $i \in \{1, \ldots, N\}$
for $t = 2$ to K **do**
    $s \leftarrow \arg\max_i \text{dist}[i]$ ▷ Furthest from current set
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}$
    for each $i \in \{1, \ldots, N\}$ **do**
        $\text{dist}[i] \leftarrow \min\big(\text{dist}[i],\ d(p_i, p_s)\big)$
    end for
end for
return $\mathcal{S}$

A.2 Clustering for Trees

This algorithm basically forms K clusters from a given angular granularity, before clustering each cluster C_k with a smaller angular granularity. This algorithm also respects temporal continuity within demonstration frames by always choosing the earliest demonstration frames to be the parent node within a cluster, and strictly choosing members of clusters to be those proceeding it. The tree is then passed to Sarkar's Constructor, which embeds each node procedurally as per Appendix C.

Algorithm 5 Cluster Function

```
function CLUSTER(cluster_idxs, θ, state, gran)
    earliest_index ← POP(cluster_idxs, 0)          ▷ Remove and get first element
    clusters ← []                                   ▷ Initialize empty list
    children ← []                                   ▷ Initialize empty list
    for idx in cluster_idxs do
        if state[earliest_index] ≠ state[idx] or |radians(θ[earliest_index] − θ[idx])| > gran
    then
            APPEND(clusters, (earliest_index, children))
            earliest_index ← idx
            children ← []
            continue
        end if
        APPEND(children, idx)
    end for
    APPEND(clusters, (earliest_index, children))
    return clusters
end function
```

B

Simplifying PointNet++

One of the issues that arose when implementing Instant Policy was training PointNet++, which is the model being used to embed spatial information within the scenes. With 3D, PointNet++ works and makes sense, as there exists multiple planes of surface, which it can utilise to embed spatial information, or abstractly the directions of vectors within a plane. However, in 2D, there exists only 1 plane globally. While there is no objective proof, I suspect that this is causing PointNet++ to not train in 2D as effectively, any clusters of points lead to very similar vector directions. As such, in the encoder-decoder training procedure, the targets become noisy, and the model fails to learn.

To overcome this, a more principled (aligned with Instant Policy) solution is to mechanically add 'texture' within object scene point clouds by adding a systematically generated and clamped 3^{rd} dimension to object scene nodes. However this can get complicated, and I opted to simplify PointNet++ instead.

To simplify Pointnet++, I kept the sampling and grouping process, whereby point clouds are first processed through Furthest Point Sampling Algorithm [31] to get M centroids. M in this case is predefined. Then by using a k-Nearest Neighbour algorithm, we group K point clouds to M centroids, and express these K point clouds in their centroid's frame.

The last step in PointNet++ usually entails a mini-PointNet layer that aims to embed vector directions for each cluster. In our case, we have swapped it out for a model that uses a 2-layer Linear layer L_p to process each point within the centroid, and a 2-layer Linear layer prediction head L_{out}that utilises aggregations from outputs of L_p and geometrical information within clusters.

Specifically, each point is embedded with fourier features (with respect to the centroid) before being passed into L_p. We then aggregate outputs from L_p with *max* and *mean*, which we will pass into L_{out} with geometrical information within cluster. Geometrical information within clusters are formally defined to be the mean distance from centroid, maximum distance from centroid and minimum distance from centroid. The final output gives us the spatial feature that attempts to encode the surrounding geometry of the centroid.

The model is trained with parameters:

K (for kNN) = 8

M (num sampled centroid) = 8

hidden dim (L_p) = $[64, 128]$

hidden dim (L_{out}) = $[64, 128]$

epoch = 10 training shown to have converge at 3+ epochs

B

Additional Augmentations : To make learning robust, jitters are added to point cloud coordinates to simulate noise.

C

Sarkar's Constructor

C.1 PseudoCode

Algorithm 6 Sarkar's Constructor

Input: Node a with parent b, children to place $c_1, c_2, \ldots, c_{\deg(a)-1}$, partial embedding f containing an embedding for a and b, scaling factor τ

$(0, z) \leftarrow \text{reflect}_{f(a)\to 0}(f(a), f(b))$

$\theta \leftarrow \arg(z)$ {angle of z from x-axis in the plane}

for $i \in \{1, \ldots, \deg(a) - 1\}$ **do**

$y_i \leftarrow \frac{e^\tau - 1}{e^\tau + 1} \cdot \left(\cos\left(\theta + \frac{2\pi i}{\deg(a)}\right), \sin\left(\theta + \frac{2\pi i}{\deg(a)}\right)\right)$

end for

$(f(a), f(b), f(c_1), \ldots, f(c_{\deg(a)-1})) \leftarrow \text{reflect}_{0\to f(a)}(0, z, y_1, \ldots, y_{\deg(x)-1})$

Output: Embedded $\mathbb{H}_2$ vectors $f(c_1), f(c_2), \ldots, f(c_{\deg(a)-1})$

C.2 Properties

C.2.1 Low distortion

Formally, [34] has proven that for any $\epsilon > 0$, by choosing scaling factor τ that is sufficiently large, the distortion embedding can be bounded by $1 + \epsilon$. In mathematical terms, for all $u, v \in V$ where

V is the set of nodes of $T = (V, E)$,

$$\frac{1}{1+\epsilon} d_T(u, v) \leq d_{\mathbb{B}}(f(u), f(v)) \leq (1+\epsilon) d_T(u, v) \tag{C.1}$$

where d_T is tree distance (path length) and $d_{\mathbb{B}}$ is the hyperbolic distance defined in Poincare ball model.

C.2.2 Root-leaf separation

Sarkar's Construction procedurally places nodes deep in the trees near boundary of the Poincare ball model, while the root stays within the centre. Since distance between root and leaves grows exponentially (since from equation D.4, denominator will be small if point is near the boundary), the root-leaf distances are maximised [34]. This also allows the embedding space to grow exponentially with the tree's branching factor.

C.2.3 Preservation of hierarchy

The reflection step in Sarkar's guarantees that children will be symmetrically distributed around the parent while maintaining separation from the parent's parent. This preserves the d_T of these children to the parent's parent, thereafter preserving the combinatorial hierarchy of the tree.

D

Hyperbolic Mathematics

Most definitions are taken from [26, 2]. If equations or definitions are not cited, they are taken to be from [26] for mobius operations, and [2] for Poincaré ball model.

D.1 Riemannian Manifold

A Riemannian manifold is defined by a tuple $(\mathcal{M}, g)$ [42], where $\mathcal{M}$ and g are the manifold and the Riemannian metric, respectively.

A manifold $\mathcal{M}$ is a set of points x that are locally similar to the linear (Euclidean) space. Every point $x \in \mathcal{M}$ has an associated tangent space $\mathcal{T}_x\mathcal{M}$, which is a real vector space of the same dimensionality as $\mathcal{M}$ and contains all the directions that can tangentially pass through x.

The Riemannian metric g defines a smoothly varying family of inner products on the tangent space.Formally:

$$g_x = \langle ., . \rangle : \mathcal{T}_x\mathcal{M} \times \mathcal{T}_x\mathcal{M}, \forall x \in \mathcal{M} \tag{D.1}$$

Each point $x \in \mathcal{M}$ is associated with a metric tensor that defines its inner product in tangent space, which induces a norm on vectors $v \in \mathcal{T}_x\mathcal{M}$

$$||v||_x := \sqrt{\langle v, v \rangle_z} \tag{D.2}$$

Hyperbolic space $\mathbb{H}$ is then a Riemannian manifold with a constant negative curvature. Typically, to define a hyperbolic space model, you will need to define $\mathcal{M}$, g and the *curvature*.

D.1.1 Poincaré Ball Model

The n-dimensional Poincaré ball model is a hyperbolic embedding model that defines a set of points in $\mathbb{R}^n$ whose Euclidean norm is less than 1

According to [26, 2], a Poincaré ball model with curvature c is defined as the manifold:

$$\mathbb{B}^n = \{x \in \mathbb{R}^n : ||x|| < \frac{1}{\sqrt{c}}\} \tag{D.3}$$

where $||\cdot||$ denotes the Euclidean norm.

The Riemannian metric tensor g_x of this manifold is given by:

$$g_x = \lambda_x^2 g^E, \ with \ \lambda_x = \frac{2}{1-c||x||^2} \tag{D.4}$$

where g^E is the Euclidean metric tensor, and λ_x is the conformal factor that scales the Euclidean metric to account for the curvature of hyperbolic space.

Within Poincaré ball model, hyperbolic distance is given by:

$$d(u,v) = \frac{1}{\sqrt{c}} \operatorname{arcosh}\left(1 + \frac{2\|u-v\|^2}{(1-\|u\|^2)(1-\|v\|^2)}\right) \tag{D.5}$$

This distance calculation is what ensures that points that are near boundaries remain exponentially far apart. This is so as when points approach boundaries, their norms $||.||$ approaches 1, which causes the denominator to approach 0. This causes the fraction to explode in magnitude, resulting in large distances.

By ensuring that points near boundaries remain far apart, Poincaré ball model makes itself suitable to embed hierarchical data [2, 30]. Treating the hierarchy within data as a tree with branching factor b and where hierarchy goes deeper down the tree, it is easy to see that at deeper levels of hierarchy, number of nodes explodes exponentially. However, since towards the boundaries, points in Poincaré ball model always remain far apart, essentially towards the boundary, we get exponentially 'more space' to embed said nodes. This then results in less distortion within embedded features.

During implementation, we take curvature $c = 1$, using the Standard Poincaré ball model for simplicity. This simplifies mobius operations, which are elaborated below, and is what most researchers choose [26, 2]. Additionally, I do not want to consider curvature as a hyper-parameter for the question, due to time constraints on the project.

D.2 Mobius Operations

Mobius Addition

For $x, y \in \mathbb{H}^n$, mobius addition $x \oplus y$ is defined as:

$$x \oplus y = \frac{(1 + 2\langle x, y\rangle + ||y||^2)x + (1 - ||x||^2)y}{1 + 2\langle x, y\rangle + ||x||^2||y||^2} \tag{D.6}$$

where $\langle ., . \rangle$ denotes Euclidean inner product.

Mobius Scalar Multiplication

For a scalar $\mathbf{r} \in \mathbb{R}$ and a vector $x \in \mathbb{H}^n$, Mobius scalar multiplication is given by:

$$\mathbf{r} \otimes x = tanh(\mathbf{r}\ tanh^{-1}(||x||))\frac{x}{||x||} \tag{D.7}$$

It is clear to see that this operation scales a vector without breaking the norm constraint for Poincaré ball model.

Mobius Matrix-Vector Multiplication

Given matrix $W \in \mathbb{R}^{mxn}$ and a vector $x \in \mathbb{H}^n$, Mobius matrix-vector multiplication $W \otimes_M x$ is:

$$W \otimes_M x = tanh(\frac{||Wx||}{||x||})tanh^{-1}(||x||))\frac{Wx}{||Wx||}, \text{if } \ x \neq 0 \tag{D.8}$$

This operation is essentially an extension of Mobius scalar multiplication on a Matrix-vector scale. However, with mobius addition and mobius Matrix-vector multiplication, it is possible to apply linear layers on Poincaré embeddings such that they respect the model's geometry. This

effectively helps performance by not breaking the geometry, we retain the ability of embeddings to capture hierarchical ability as a hyperbolic space.

E

Figures

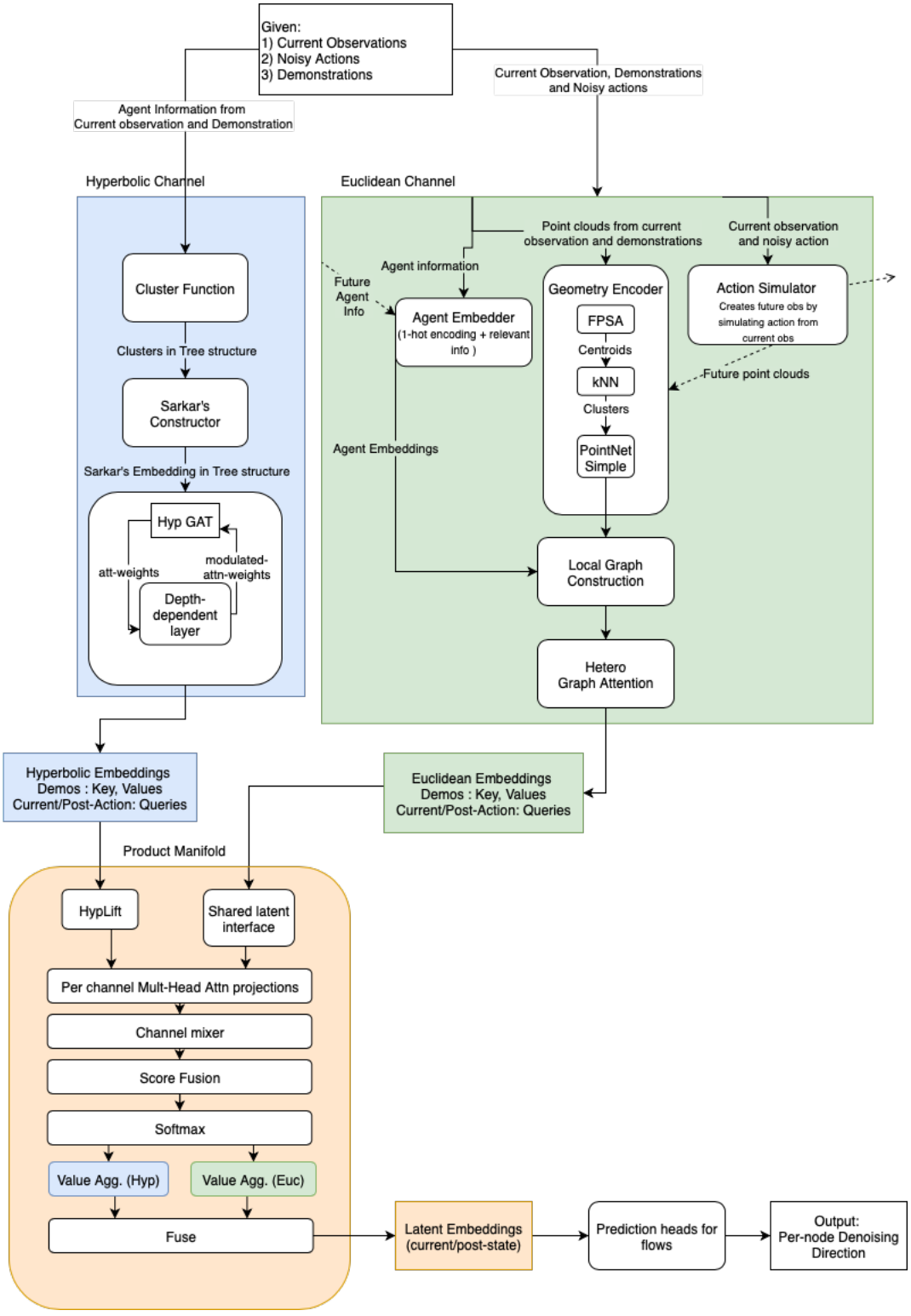


Figure E.1: Agent Architecture (Detailed)

Bibliography

1. Vosylius V and Johns E. Instant Policy: In-Context Imitation Learning via Graph Diffusion. 2025. arXiv: 2411.12633 [cs.RO]. Available from: https://arxiv.org/abs/2411.12633

2. Nickel M and Kiela D. Poincaré Embeddings for Learning Hierarchical Representations. 2017. arXiv: 1705.08039 [cs.AI]. Available from: https://arxiv.org/abs/1705.08039

3. Bansal S and Benton A. Comparing Euclidean and Hyperbolic Embeddings on the WordNet Nouns Hypernymy Graph. 2021. arXiv: 2109.07488 [cs.CL]. Available from: https://arxiv.org/abs/2109.07488

4. Dong H, Zhang J, and Zhang C. Leveraging Hyperbolic Embeddings for Coarse-to-Fine Robot Design. 2023. arXiv: 2311.00462 [cs.AI]. Available from: https://arxiv.org/abs/2311.00462

5. Jaquier N, Rozo L, González-Duque M, Borovitskiy V, and Asfour T. Bringing motion taxonomies to continuous domains via GPLVM on hyperbolic manifolds. 2024. arXiv: 2210.01672 [cs.RO]. Available from: https://arxiv.org/abs/2210.01672

6. Moreira G, Marques M, Costeira JP, and Hauptmann A. Hyperbolic vs Euclidean Embeddings in Few-Shot Learning: Two Sides of the Same Coin. 2023. arXiv: 2309.10013 [cs.CV]. Available from: https://arxiv.org/abs/2309.10013

7. Tsuji T, Kato Y, Solak G, Zhang H, Petrič T, Nori F, and Ajoudani A. A Survey on Imitation Learning for Contact-Rich Tasks in Robotics. 2025. arXiv: 2506.13498 [cs.RO]. Available from: https://arxiv.org/abs/2506.13498

8. Argall BD, Chernova S, Veloso M, and Browning B. A survey of robot learning from demonstration. Robotics and Autonomous Systems 2009; 57:469–83. DOI: https://doi.org/10.1016/j.robot.2008.10.024. Available from: https://www.sciencedirect.com/science/article/pii/S0921889008001772

9. Ravichandar H, Polydoros A, Chernova S, and Billard A. Recent Advances in Robot Learning from Demonstration. Annual Review of Control, Robotics, and Autonomous Systems 2020 May; 3. DOI: 10.1146/annurev-control-100819-063206

10. Radosavovic I, Wang X, Pinto L, and Malik J. State-Only Imitation Learning for Dexterous Manipulation. 2021. arXiv: 2004.04650 [cs.RO]. Available from: https://arxiv.org/abs/2004.04650

11. Yu T, Finn C, Xie A, Dasari S, Zhang T, Abbeel P, and Levine S. One-Shot Imitation from Observing Humans via Domain-Adaptive Meta-Learning. 2018. arXiv: 1802.01557 [cs.LG]. Available from: https://arxiv.org/abs/1802.01557

12. Caron M, Touvron H, Misra I, Jégou H, Mairal J, Bojanowski P, and Joulin A. Emerging Properties in Self-Supervised Vision Transformers. 2021. arXiv: 2104.14294 [cs.CV]. Available from: https://arxiv.org/abs/2104.14294

13. Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, and Amodei D. Language Models are Few-Shot Learners. 2020. arXiv: 2005.14165 [cs.CL]. Available from: https://arxiv.org/abs/2005.14165

14. Parnami A and Lee M. Learning from Few Examples: A Summary of Approaches to Few-Shot Learning. 2022. arXiv: 2203.04291 [cs.LG]. Available from: https://arxiv.org/abs/2203.04291

15. Fu L, Huang H, Datta G, Chen LY, Panitch WCH, Liu F, Li H, and Goldberg K. In-Context Imitation Learning via Next-Token Prediction. 2024. arXiv: 2408.15980 [cs.RO]. Available from: https://arxiv.org/abs/2408.15980

16. Gu A, Sala F, Gunel B, and Ré C. Learning Mixed-Curvature Representations in Product Spaces. *International Conference on Learning Representations*. 2019. Available from: https://openreview.net/forum?id=HJxeWnCcF7

17. Zheng W, Wang W, Zhao S, and Qian F. Hyperbolic Hierarchical Knowledge Graph Embeddings for Link Prediction in Low Dimensions. 2024. arXiv: 2204.13704 [cs.LG]. Available from: https://arxiv.org/abs/2204.13704

18. Wang S, Wei X, Nogueira dos Santos CN, Wang Z, Nallapati R, Arnold A, Xiang B, Yu PS, and Cruz IF. Mixed-Curvature Multi-Relational Graph Neural Network for Knowledge Graph Completion. *Proceedings of the Web Conference 2021*. WWW '21. Ljubljana, Slovenia: Association for Computing Machinery, 2021 :1761–71. DOI: 10.1145/3442381.3450118. Available from: https://doi.org/10.1145/3442381.3450118

19. Bordes A, Usunier N, Garcia-Duran A, Weston J, and Yakhnenko O. Translating Embeddings for Modeling Multi-relational Data. *Advances in Neural Information Processing Systems*. Ed. by Burges C, Bottou L, Welling M, Ghahramani Z, and Weinberger K. Vol. 26. Curran Associates, Inc., 2013. Available from: https://proceedings.neurips.cc/paper_files/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf

20. Sun Z, Deng ZH, Nie JY, and Tang J. RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space. 2019. arXiv: 1902.10197 [cs.LG]. Available from: https://arxiv.org/abs/1902.10197

21. Ungar AA. Hyperbolic trigonometry and its application in the Poincaré ball model of hyperbolic geometry. Computers & Mathematics With Applications 2001; 41:135–47. Available from: https://api.semanticscholar.org/CorpusID:14866011

22. Mishne G, Wan Z, Wang Y, and Yang S. The numerical stability of hyperbolic representation learning. *Proceedings of the 40th International Conference on Machine Learning*. ICML'23. Honolulu, Hawaii, USA: JMLR.org, 2023

23. Gulcehre C, Denil M, Malinowski M, Razavi A, Pascanu R, Hermann KM, Battaglia P, Bapst V, Raposo D, Santoro A, and Freitas N de. Hyperbolic Attention Networks. 2018. arXiv: 1805.09786 [cs.NE]. Available from: https://arxiv.org/abs/1805.09786

24. Shimizu R, Mukuta Y, and Harada T. Hyperbolic Neural Networks++. 2021. arXiv: 2006.08210 [cs.LG]. Available from: https://arxiv.org/abs/2006.08210

25. Yang M, Verma H, Zhang DC, Liu J, King I, and Ying R. Hypformer: Exploring Efficient Transformer Fully in Hyperbolic Space. 2025. arXiv: 2407.01290 [cs.LG]. Available from: https://arxiv.org/abs/2407.01290

26. Fein-Ashley J, Feng E, and Pham M. HVT: A Comprehensive Vision Framework for Learning in Non-Euclidean Space. 2024. arXiv: 2409.16897 [cs.CV]. Available from: https://arxiv.org/abs/2409.16897

27. Yang M, Zhou M, Kalander M, Huang Z, and King I. Discrete-time Temporal Network Embedding via Implicit Hierarchical Learning in Hyperbolic Space. *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. KDD '21. ACM, 2021 Aug :1975–85. DOI: 10.1145/3447548.3467422. Available from: http://dx.doi.org/10.1145/3447548.3467422

28. Sankar A, Wu Y, Gou L, Zhang W, and Yang H. Dynamic Graph Representation Learning via Self-Attention Networks. 2019. arXiv: 1812.09430 [cs.LG]. Available from: https://arxiv.org/abs/1812.09430

29. Jensen KT, Kao TC, Tripodi M, and Hennequin G. Manifold GPLVMs for discovering non-Euclidean latent structure in neural data. 2020. arXiv: 2006.07429 [stat.ML]. Available from: https://arxiv.org/abs/2006.07429

30. Sala F, De Sa C, Gu A, and Re C. Representation Tradeoffs for Hyperbolic Embeddings. *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Dy J and Krause A. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018 Jul :4460–9. Available from: https://proceedings.mlr.press/v80/sala18a.html

31. Qi CR, Yi L, Su H, and Guibas LJ. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. 2017. arXiv: 1706.02413 [cs.CV]. Available from: https://arxiv.org/abs/1706.02413

32. Mescheder L, Oechsle M, Niemeyer M, Nowozin S, and Geiger A. Occupancy Networks: Learning 3D Reconstruction in Function Space. 2019. arXiv: 1812.03828 [cs.CV]. Available from: https://arxiv.org/abs/1812.03828

33. Zhou A, Kim MJ, Wang L, Florence P, and Finn C. NeRF in the Palm of Your Hand: Corrective Augmentation for Robotics via Novel-View Synthesis. 2023. arXiv: 2301.08556 [cs.LG]. Available from: https://arxiv.org/abs/2301.08556

34. Sarkar R. Low distortion delaunay embedding of trees in hyperbolic plane. *Proceedings of the 19th International Conference on Graph Drawing*. GD'11. Eindhoven, The Netherlands: Springer-Verlag, 2011 :355–66. DOI: 10.1007/978-3-642-25878-7_34. Available from: https://doi.org/10.1007/978-3-642-25878-7_34

35. Qu H, Song YR, Zhang M, Jiang GP, Li R, and Song B. Ha-gnn: a novel graph neural network based on hyperbolic attention. Neural Computing and Applications 2024 Apr; 36:1–16. DOI: 10.1007/s00521-024-09689-9

36. Karcher H. Riemannian center of mass and mollifier smoothing. Communications on Pure and Applied Mathematics 1977; 30:509–41. DOI: https://doi.org/10.1002/cpa.3160300502. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpa.3160300502. Available from: https://onlinelibrary.wiley.com/doi/abs/10.1002/cpa.3160300502

37. Yamazaki T. kurims.kyoto-u.ac.jp. https://www.kurims.kyoto-u.ac.jp/~kyodo/kokyuroku/contents/pdf/1839-05.pdf. 2013

38. Zhang Y, Wang X, Jiang X, Shi C, and Ye Y. Hyperbolic Graph Attention Network. 2019. arXiv: 1912.03046 [cs.LG]. Available from: https://arxiv.org/abs/1912.03046

39. Liu Q, Nickel M, and Kiela D. Hyperbolic Graph Neural Networks. 2019. arXiv: 1910.12892 [cs.LG]. Available from: https://arxiv.org/abs/1910.12892

40. Johns E. Coarse-to-Fine Imitation Learning: Robot Manipulation from a Single Demonstration. 2021. arXiv: 2105.06411 [cs.RO]. Available from: https://arxiv.org/abs/2105.06411

41. Ho J, Jain A, and Abbeel P. Denoising Diffusion Probabilistic Models. 2020. arXiv: 2006.11239 [cs.LG]. Available from: https://arxiv.org/abs/2006.11239

42. Peterson P. Riemannian Geometry — link.springer.com. https://link.springer.com/book/10.1007/978-0-387-29403-2. 2016